



Preface

Every connected vehicle is now a software platform on wheels — one with real-time network connectivity, cloud-linked services, over-the-air update capability, and an ever-expanding set of external interfaces. The same features that make modern vehicles useful also make them attackable. A researcher in 2015 demonstrated this with a remote compromise of a Jeep Cherokee from an internet connection. By 2023, a vulnerability in a manufacturer's cloud API allowed unauthorized remote start and location tracking of millions of vehicles using only an email address. The attack surface has only grown since.

This handbook is a practitioner's reference for automotive cybersecurity engineering. It covers the full lifecycle — from threat analysis at concept phase through secure architecture, cryptographic controls, over-the-air updates, detection and monitoring, incident response, and end-of-life decommissioning. It integrates the primary regulatory and standards framework: ISO/SAE 21434, UNECE R155 and R156, ISO 26262, EU Cyber Resilience Act, and NIST Post-Quantum Cryptography standards.

The handbook is organized in five parts. Part I establishes the business and regulatory case for cybersecurity as a first-class engineering discipline. Part II covers engineering practice from threat analysis through secure development and testing. Part III addresses field operations, detection, and incident response. Part IV examines emerging domains — EV charging, V2X, ADAS, SDV architecture, AI/ML systems, and privacy. Part V provides reference material including readiness checklists, a maturity model, quick-reference sheets, and an extensive glossary.

This is version 1.0, updated June 13, 2026. It incorporates corrections to the post-quantum cryptography section reflecting NIST's finalization of FIPS 203, 204, and 205 in August 2024; updated EU CRA enforcement timelines; a complete ASIL determination matrix; and two new appendices covering the automotive cybersecurity glossary and post-quantum standards reference.

All case studies in this handbook are based on publicly disclosed research. No proprietary OEM data, confidential supplier specifications, or unpublished vulnerability information is included.

Ashish Kumar

Senior Technical Leader & Associate Manager

KPIT Technologies Limited, Bengaluru

June 2026

TABLE OF CONTENTS

Version History&Enhancement Notes

Preface

PART I — BUSINESS CASE®ULATORY CONTEXT

- Chapter 1: Why Automotive Cybersecurity Matters
- Chapter 2: Vehicle Architecture and Attack Surface
- Chapter 3: Threat Landscape and Attack Chains
- Chapter 4: Standards and Regulations

PART II — ENGINEERING PRACTICE

- Chapter 5: Secure Product Lifecycle
- Chapter 6: TARA — Threat Analysis and Risk Assessment
- Chapter 7: HARA and Safety-Security Integration
- Chapter 8: Secure Architecture Principles
- Chapter 9: Secure Boot, Cryptography, and Identity
- Chapter 10: Secure OTA Update Pipeline
- Chapter 11: Secure Development and Security Testing

PART III — FIELD OPERATIONS

- Chapter 12: Detection, Logging, and Anomaly Monitoring
- Chapter 13: Incident Response Playbook

PART IV — EMERGING DOMAINS

- Chapter 14: Special Domains&Cross-Cutting Concerns

PART V — REFERENCE MATERIAL

- Chapter 15: OEM and Supplier Readiness Checklist
- Chapter 16: Maturity Roadmap and Common Mistakes
- Chapter 17: Quick Reference Summary and Cheat Sheet

APPENDICES

- Appendix A: Jeep Cherokee Remote Hack (2015)
- Appendix B: Tesla Security Program — Industry Benchmark
- Appendix C: Kia / Hyundai Connected Vehicle API Vulnerabilities (2022-2023)
- Appendix D: Automotive Software Supply Chain Security
- Appendix E: Software Defined Vehicle (SDV) Security
- Appendix F: Advanced AI / ML Security for Automotive Systems
- Appendix G: Cybersecurity Tool Reference Guide
- Appendix H: Automotive Cybersecurity Maturity Model
- Appendix I: Automotive Cybersecurity Glossary
- Appendix J: Post-Quantum Cryptography Standards Reference (FIPS 203/204/205)

PART I

BUSINESS CASE®ULATORY CONTEXT

Why security matters, the vehicle attack surface, threats, and the standards ecosystem.

Chapter 1: Why Automotive Cybersecurity Matters

Chapter 2: Vehicle Architecture and Attack Surface

Chapter 3: Threat Landscape and Attack Chains

Chapter 4: Standards and Regulations

Chapter 1: Why Automotive Cybersecurity Matters

Connected vehicles are software-rich cyber-physical systems. A single cyber incident can escalate into a safety event affecting occupants, other road users, and the brand. This chapter establishes the foundational case for treating cybersecurity as a first-class engineering discipline — not a compliance checkbox or a late-stage retrofit.

1.1 The Modern Vehicle Reality

Contemporary vehicles contain hundreds of Electronic Control Units (ECUs), gigabytes of onboard software, wireless connectivity stacks, and cloud-linked telematics. Attack paths multiply with every new interface:

- Bluetooth, USB, Wi-Fi, cellular (4G/5G), OBD-II, OTA update channels, V2X communications, and EV charging ports each constitute an external attack surface.
- Software-Defined Vehicles (SDVs) increase feature delivery velocity but also expand software complexity, supply-chain dependency, and the consequence of a single-node compromise.
- The same compute platform must simultaneously satisfy safety, privacy, reliability, and regulatory compliance requirements — creating complex engineering trade-offs.
- Vehicles remain in service for 10 to 20 years, outlasting typical cryptographic algorithm lifetimes and software framework support windows.
- Feature parity now requires cloud connectivity. The cloud infrastructure managing millions of vehicles introduces a backend attack surface with fleet-wide blast radius.

1.2 What Can Go Wrong

The consequences of a successful automotive cyber attack fall into three broad categories:

Safety Consequences

Loss of braking, steering, or propulsion control; sensor spoofing leading to incorrect ADAS decisions; false actuator commands bypassing driver intent; manipulation of autonomous driving systems.

Operational Consequences

Ransomware locking vehicle systems or infotainment; denial-of-service on gateway or TCU; corrupted or unauthorized OTA updates rendering ECUs inoperative; fleet-wide service disruption from a compromised backend.

Business Consequences

Recall campaigns, certification delays, warranty exposure, regulatory sanctions, and brand reputation damage. The first software-only vehicle recall in US history — affecting 1.4 million vehicles — was triggered by a cybersecurity vulnerability in 2015.

1.3 Three Core Principles

Principle	Engineering Meaning
Design Security In	Security requirements, threat models, and trust boundaries must be established at concept phase, not retrofitted at release. The cost of a security fix grows 10× for every lifecycle phase it is deferred.
Detect Early	Anomaly monitoring, IDS, and logging allow teams to see compromise before it causes harm. An unmonitored fleet is an invisible fleet.

Principle	Engineering Meaning
Recover Safely	Fail-safe modes, rollback mechanisms, and incident response plans ensure vehicles can recover securely. A tested recovery path is as important as the primary happy path.

1.4 Business and Regulatory Impact

- **UNECE R155** mandates a Cyber Security Management System (CSMS) for vehicle type approval in the EU, Japan, South Korea, India, and other signatory markets.
- **UNECE R156** mandates a Software Update Management System (SUMS) for controlled OTA and software update governance.
- **EU Cyber Resilience Act (2024)** extends cybersecurity obligations to all products with digital elements placed on the EU market, entering into force December 2024.
- Non-compliance with R155/R156 results in denied or revoked vehicle type approval — meaning no vehicle sales in signatory markets.
- Security readiness has become a product competitiveness factor and a procurement criterion for fleet buyers, not merely a compliance checkbox.

i KEY PRINCIPLE

Treat cybersecurity like engineering: requirements, architecture controls, test evidence, and release gates. Not a last-minute audit.

Chapter 2: Vehicle Architecture and Attack Surface

Understanding the vehicle as a connected cyber-physical system is prerequisite to identifying where risk lives. This chapter maps the architecture from external touchpoints to core safety functions, and surveys the in-vehicle networks that carry security-critical traffic.

2.1 Architecture Evolution

- **Legacy vehicles** used isolated domain ECUs with minimal external connectivity. Security was implicit in physical access barriers.
- **Connected vehicles** introduced telematics, cloud sync, mobile apps, and OTA paths — creating persistent internet-reachable interfaces.
- **Zonal and domain-controller architectures** reduce wiring harness mass but increase the importance of gateways and shared compute nodes.
- **Software-Defined Vehicles (SDVs)** decouple software from hardware. This means a single compromised compute node can simultaneously affect multiple vehicle functions.
- **2026 SDV platforms** typically include a high-performance compute (HPC) node running Linux or QNX, a hypervisor separating safety and infotainment domains, and SOME/IP service-oriented middleware.

2.2 Vehicle Trust Model (Layered View)

Layer	Zone	Contents
1	External entry points	Mobile app, cloud backend, USB, OBD-II, Bluetooth, Wi-Fi, cellular, V2X, EV charging interface
2	Control plane	TCU, gateway ECU, authentication service, update server, diagnostic tools, SOVD
3	Vehicle domains	Powertrain, chassis, body, infotainment, ADAS, battery management, HMI
4	Safety-critical core	Actuators, sensors, motion control, energy delivery, fallback behavior, safe state logic

1 KEY PRINCIPLE

Every external interface is a possible entry point and must be treated as a controlled trust boundary. Compromise at layer 1 can, if not stopped by the control plane, propagate to layer 4.

2.3 Core Building Blocks

ECU (Electronic Control Unit)

A specialized embedded controller managing one or more vehicle functions. Modern ECUs may run AUTOSAR Classic or Adaptive stacks.

Domain / Zonal Controller

An ECU consolidating multiple related functions, often with an integrated HSM. Zonal controllers are organized by physical vehicle zone rather than function.

Gateway ECU

The primary security and routing boundary between network domains or vehicle zones. It enforces message filtering, authentication, and rate limiting.

HSM (Hardware Security Module)

Dedicated hardware for secure key operations, MAC generation, and protected storage. Required for SecOC and secure boot in safety-critical ECUs.

TCU (Telematics Control Unit)

The main remote connectivity bridge via cellular network. Typically the vehicle's primary external attack surface due to its persistent internet-reachable interface.

HPC (High-Performance Compute Node)

Central compute platform in SDV architectures running multiple guest VMs or containers. A compromise here has wider blast radius than a single function ECU.

2.4 In-Vehicle Network Comparison

Network	Typical Use	Strengths	Security Concerns	Common Protection
CAN / CAN-FD	Powertrain, body, chassis	Simple, robust, deterministic	Broadcast bus, no native auth, replay, injection	Gateway filtering, freshness counter, IDS
LIN	Low-cost body devices	Cheap, lightweight	Weak trust, limited bandwidth	Master-slave control, network isolation
FlexRay	High-reliability control (X-by-wire)	Deterministic, fault tolerant	Smaller ecosystem, gateway exposure	MAC, segmentation, static scheduling
Automotive Ethernet (100/1G)	Backbone, ADAS, infotainment	High bandwidth, scalable	IP attacks, lateral movement, discovery	VLANs, firewall rules, TLS, certificates
SOME/IP	Service-oriented features	Flexible service model	Unauthorized service invocation, enumeration	Mutual auth, access control, secure APIs
BLE / Wi-Fi / Cellular	External connectivity, OTA	User convenience, OTA delivery	Pairing abuse, replay, rogue endpoints	Secure pairing, rate limits, monitoring
V2X (C-V2X / DSRC)	Cooperative awareness	Safety, traffic info	PKI complexity, replay, misbehavior	PKI, pseudonymous certs, MBMS
ISO 15118 (EV Charging)	V2G, Plug&Charge	Encrypted auth, billing integration	Charger as entry point, billing fraud	TLS 1.3, ECC certificates, DH key exchange

2.5 External Interface Attack Surface

- **Bluetooth and Wi-Fi** expose pairing protocols, device discovery, and local network services to proximity attackers.
- **USB and media ports** may permit code introduction, data exfiltration, or service protocol abuse via malicious media.
- **Cellular and cloud APIs** expose session handling, confused-deputy patterns, and backend trust relationships.
- **OBD-II and diagnostic ports** are powerful service interfaces and must be strongly governed via session authentication and role-based access.
- **EV Charging and V2G interfaces** (ISO 15118) introduce energy, billing, and protocol dependencies with external parties that may be compromised.
- **Third-party applications** (Android Automotive, app stores) extend the vehicle perimeter beyond the physical car into the software supply chain.

- **Over-The-Air (OTA) update channels** are a vector for firmware tampering if signing and integrity verification are not enforced end-to-end.

Chapter 3: Threat Landscape and Attack Chains

Threats are most useful when understood as chains, not isolated weaknesses. A single exposed interface is rarely the objective; attackers use it to pivot toward high-impact assets. This chapter covers common attack patterns, threat taxonomy, and supply chain risk.

3.1 Remote Attack Chain Model

Most consequential automotive attacks follow a five-step chain:

- 1. **Discover:** Identify exposed services, APIs, ports, or weak pairing flows via passive reconnaissance or active scanning.
- 2. **Enter:** Authenticate poorly (e.g., token replay), exploit a parser, or abuse a trust relationship (e.g., OBD session elevation).
- 3. **Persist:** Maintain access via credentials, backdoors, or update-path compromise. Persistence mechanisms must survive ECU resets.
- 4. **Elevate:** Pivot from infotainment or TCU to safety-relevant domains by exploiting insufficient gateway segmentation.
- 5. **Impact:** Modify vehicle behavior, block service, exfiltrate data, or degrade safety functions.

1 KEY PRINCIPLE

A vulnerability in infotainment has limited direct impact. Its risk multiplies if no gateway segmentation prevents pivoting to powertrain or chassis ECUs.

3.2 Threat Taxonomy (STRIDE)

Category	Examples	Objective Affected	Primary Controls
Spoofing	Forged ECU identity, fake credentials, malicious pairing, API token replay	Authentication	PKI certificates, MAC, challenge-response, mutual TLS
Tampering	Modified firmware, altered CAN messages, route hijack, OTA package swap	Integrity	Secure boot, signed images, CMAC, CRC, anti-rollback
Repudiation	Missing logs, disputed commands, weak trace, clock manipulation	Accountability	Tamper-evident logs, timestamps, audit trails, secure time source
Information Disclosure	Data exfiltration, key leakage, location history, PII exposure	Confidentiality	Encryption, least privilege, data minimization, GDPR controls
Denial of Service	Bus flooding, RF jamming, resource exhaustion, ransomware lock	Availability	Rate limiting, watchdog timers, fallback modes, network isolation
Elevation of Privilege	Sandbox escape, diagnostic abuse, trust pivoting, session escalation	Authorization	Least privilege, segmentation, session control, role-based access

3.3 Common Attack Vectors

Remote Network Abuse

- Cloud API abuse through weak authorization, token leakage, or confused-deputy behavior — particularly Broken Object-Level Authorization (BOLA).
- Telematics unit vulnerabilities in message parsing, trust management, or session handling over cellular.
- Infotainment exploitation through media parsers, browser engines (WebKit/Chromium), or exposed local services.

Physical and Service Tool Abuse

- OBD-II misuse, unauthorized diagnostic session escalation, or memory dump via debug ports left active in production.
- Firmware extraction from JTAG, UART, SWD interfaces left accessible in production hardware.
- ECU replacement attacks, in-line harness injection, or key cloning via proximity to provisioning infrastructure.

Wireless and Proximity Attacks

- BLE pairing abuse; relay attacks on keyless entry systems (PEPS — Passive Entry Passive Start) using signal amplification equipment available for under \$100.
- Wi-Fi deauthentication and evil-twin attacks on vehicles with built-in hotspot functionality.
- V2X message injection, GPS spoofing, or misbehavior detection evasion.
- NFC-based digital key relay attacks targeting vehicles using Ultra-Wideband (UWB) or NFC key solutions.

3.4 Software Supply Chain Attacks

A critical threat vector that bypasses perimeter controls entirely by targeting the software build and delivery pipeline:

- **Compromised open-source libraries** enter via code reuse, package registries (npm, PyPI, Conan), or vendor toolchains. The XZ Utils backdoor (CVE-2024-3094, March 2024) demonstrated how a targeted multi-year supply chain attack can compromise widely deployed infrastructure.
- **Build pipeline compromise** can alter binaries without changing source code or triggering review alerts.
- **Signing infrastructure** is a prime target because compromise enables trusted distribution of malicious software to the entire fleet.
- **Software Bill of Materials (SBOM)** in CycloneDX or SPDX format is the foundational tool for supply chain visibility and rapid CVE impact assessment.

3.5 2024–2026 Threat Intelligence Highlights

- **LLM-assisted vulnerability discovery:** AI tools are being used by both security researchers and threat actors to accelerate automotive ECU firmware analysis and CAN traffic pattern analysis.
- **Automotive ransomware campaigns:** Post-2022 incidents targeting fleet management backends and dealer management systems, with cascading impact on vehicle operations.
- **EV charging infrastructure attacks:** Growing attack surface as Plug & Charge (ISO 15118) deployment scales, including backend billing fraud and charger-as-entry-point scenarios.

- **Digital key attacks:** With ISO 18013-5 and UWB-based digital keys proliferating, relay and replay attacks targeting digital car keys have become a documented threat category.

3.6 Attack Surface by Lifecycle Stage

Stage	Attack Surface
Concept	Requirements gaps, architecture assumptions, insecure interface choices, missing TARA scope
Development	Source code, build systems, test tools, supplier artifacts, CI/CD pipelines, dependency registries
Production	Programming, provisioning, calibration, key injection, logistics chain, factory network
Operation	Remote services, cloud APIs, user mobile apps, OTA update campaigns, V2X infrastructure
Service	Dealer/workshop tools, parts replacement, ECU reflashing, service credentials, SOVD access
Decommission	Data wiping, key revocation, component resale or disposal, recycler access

I KEY PRINCIPLE

Always ask four questions: What can be attacked? What is the worst-case outcome? How would we detect it? How do we recover safely?

Chapter 4: Standards and Regulations

Engineering guidance and regulatory requirements work together. Standards define the how; regulations define the must. A product team that understands both can build traceable evidence efficiently and avoid costly rework at approval time.

4.1 ISO/SAE 21434:2021 — Road Vehicle Cybersecurity Engineering

- Defines engineering requirements for cybersecurity risk management across the full road vehicle lifecycle.
- Covers concept, product development, production, operation, maintenance, and decommissioning.
- Focuses on work products, traceability, risk treatment decisions, and evidence-based engineering.
- Introduces Cybersecurity Assurance Levels (CAL 1 to CAL 4) analogous to ASIL in ISO 26262.
- Defines TARA (Threat Analysis and Risk Assessment) methodology as the primary risk tool.
- Requires a Cybersecurity Management System (CSMS) at the organizational level.
- Addresses distributed development and supplier security requirements (Clause 7).

4.2 UNECE UN Regulation No. 155 (R155)

- Mandates implementation of a Cyber Security Management System (CSMS) for vehicle type approval.
- Covers risk identification, mitigation, monitoring, response, and continuous improvement across the vehicle lifecycle.
- Applies to M, N, O, and L6e/L7e vehicle categories in signatory markets (EU, Japan, South Korea, India, others).
- OEMs must demonstrate that their CSMS is independently audited and certified.
- Suppliers must receive security requirements flowed down from OEM contracts.
- **WP.29 Phase 2 (2025–2026):** The United Nations Working Party on Automated/Autonomous and Connected Vehicles (GRVA) is developing enhanced requirements to address SDV architectures, AI-enabled systems, and expanded interface coverage.

4.3 UNECE UN Regulation No. 156 (R156)

- Mandates a Software Update Management System (SUMS) covering controlled software update delivery.
- Requires integrity, authenticity, compatibility, traceability, and campaign control for all software updates.
- Covers OTA updates, workshop updates, and field software maintenance.
- Anti-rollback, customer communication, and update campaign evidence are explicit requirements.
- **2026 scope expansion:** SUMS requirements are being extended to cover ML model updates in ADAS systems, creating a direct link to SOTIF compliance activities.

4.4 NHTSA Cybersecurity Best Practices (2022; Refreshed 2024)

- Non-binding US guidance emphasizing risk-based cybersecurity, supply chain attention, and incident response.
- **2024 refresh** introduced stronger language on: SBOM requirements, OTA security, cloud API governance, and EV charging interface security.
- Encourages governance, information sharing (Auto-ISAC participation), and security-by-design.
- Supports continuous improvement rather than compliance point-in-time.
- Increasingly referenced in US federal procurement and NIST Cybersecurity Framework 2.0 alignment.

4.5 EU Cyber Resilience Act (CRA) — 2024 Status&Automotive Implications

⚠ NOTE

The EU CRA entered into force on December 11, 2024. Manufacturer obligations apply from December 11, 2027 (36-month transition period). Notification body and market surveillance requirements apply from June 11, 2026.

- Products with digital elements placed on the EU market must meet essential cybersecurity requirements.
- Manufacturers must handle vulnerabilities and provide security updates for the **expected product lifetime** — up to 10 years for vehicles.
- **Reporting obligation:** Actively exploited vulnerabilities must be reported to ENISA within 24 hours of discovery.
- **Conformity assessment:** Class II products (safety-relevant) require third-party assessment; vehicles with UNECE R155 certification may qualify for aligned assessment paths.
- Overlap with R155 for vehicles: compliance strategies need to be aligned to avoid double evidence burden. ENISA and WP.29 are coordinating harmonization guidance.
- **Software updates:** CRA requires security update support for the reasonably expected product lifetime; for vehicles, this is typically aligned with the planned service life.

4.6 ISO 21448:2022 — SOTIF (Safety of the Intended Functionality)

- Addresses safety risks from performance limitations or foreseeable misuse of systems where no malfunction is present — critical for ADAS and autonomous driving.
- **Cybersecurity-SOTIF intersection (2026):** Adversarial inputs to perception systems (adversarial ML attacks) can trigger SOTIF-relevant scenarios via deliberate external manipulation.
- Engineering teams must coordinate TARA and SOTIF analysis for AI/ML-enabled ADAS functions; an adversarial attack achieving a SOTIF failure requires ASIL and CAL treatment.
- SOTIF validation includes edge cases and out-of-distribution inputs — overlapping with adversarial robustness testing required by ISO/SAE 21434 for AI components.

4.7 Standards-to-Artifacts Mapping

Domain	Typical Evidence	Why It Matters
Governance	Policies, roles, training records, management reviews	Demonstrates organizational control and CSMS maturity

Domain	Typical Evidence	Why It Matters
Risk Engineering	TARA records, HARA inputs, cybersecurity goals, risk decisions	Shows reasoned, traceable risk treatment
Design	Architecture diagrams, trust boundaries, security requirements, SecOC config	Proves security was built in, not added late
Verification	Test plans, results, traceability matrices, pen test reports, defect closure	Demonstrates that security controls were actually tested
Operations	Logs, alerts, patch campaigns, incident records, IDS tuning logs	Proves the vehicle is managed post-release
Supply Chain	SBOM, supplier attestations, code signing evidence, TARA inputs	Limits hidden dependency and integration risk

i KEY PRINCIPLE

Every cybersecurity requirement should be mapped to evidence, owners, and release gates. "We plan to do it" is not evidence. Work products and traceability are what auditors and approval authorities want to see.

PART II

ENGINEERING PRACTICE

Lifecycle security, TARA, HARA, cryptography, OTA, secure development, and testing.

Chapter 5: Secure Product Lifecycle

Chapter 6: TARA — Threat Analysis and Risk Assessment

Chapter 7: HARA and Safety-Security Integration

Chapter 8: Secure Architecture Principles

Chapter 9: Secure Boot, Cryptography, and Identity

Chapter 10: Secure OTA Update Pipeline

Chapter 11: Secure Development and Security Testing

Chapter 5: Secure Product Lifecycle

Security is an end-to-end lifecycle activity, not a single test phase. ISO/SAE 21434 structures security engineering across seven major lifecycle phases. Each phase must produce traceable work products and security evidence.

5.1 Lifecycle Phases and Security Activities

#	Phase	Key Activities	Evidence Required
1	Concept	Asset inventory, use cases, trust boundaries, cybersecurity goals, initial TARA scope	Cybersecurity concept, item definition, interface registry
2	Design	TARA, architecture controls, security requirements, interface specs, HSM integration plan	TARA report, cybersecurity requirements, architecture diagram
3	Implementation	Secure coding, key handling, hardening, HSM integration, SecOC configuration	Code review records, SAST reports, coding standard compliance
4	Verification	Fuzzing, pen test, HIL, abuse cases, traceability closure	Test plans, results, finding tracker, coverage report
5	Production	Provisioning, key injection, signing, secure manufacturing controls	Provisioning records, signing evidence, factory test records
6	Operations	Monitoring, patching, incident response, fleet management, CVE triage	Incident records, patch evidence, IDS logs, SBOM CVE mapping
7	End of Life	Decommission, data wiping, key revocation, component disposal	Data deletion evidence, key revocation records

5.2 Traceability Chain

Element	Content and Link
Asset	What must be protected and where it resides (ECU flash, cloud storage, workshop tool)
Threat	How the asset can be compromised and by what attacker model (remote, physical, supply chain)
Risk	Why this threat merits attention now (impact × feasibility = risk value)
Goal	The protection outcome required (Cybersecurity Goal, assigned CAL)
Requirement	The specific engineering requirement derived from the goal (testable, owned, traceable)
Verification	How the requirement is tested, with result, acceptance criteria, and pass/fail evidence

i KEY PRINCIPLE

Keep one threat model alive across the product lifecycle. Update it whenever the architecture, supplier deliverables, or vehicle interfaces change. Evidence should be traceable from asset

through threat, risk decision, requirement, design control, and test result.

5.3 Distributed Development — Clause 7 Obligations

When cybersecurity work is distributed across OEM and supplier tiers:

- **Security Interface Agreements (SIA):** Define which party performs which security activities, what evidence each party produces, and how evidence is shared.
- **Supplier TARA:** Tier-1 and Tier-2 suppliers must perform TARA on their deliverables; the OEM integrates supplier risk information into the system-level TARA.
- **SBOM delivery:** Suppliers must provide SBOM for all software components, including open-source libraries, with version and license information.
- **Vulnerability disclosure:** Suppliers must have a vulnerability disclosure process aligned with the OEM's incident response timeline.
- **Evidence review:** OEMs must review and accept supplier security evidence as part of the integration gate.

Chapter 6: TARA — Threat Analysis and Risk Assessment

TARA is the structured method defined in ISO/SAE 21434 to identify what matters most, how it can be attacked, how damaging the impact would be, and what to do about it. It is the engine of automotive cybersecurity engineering.

6.1 TARA Workflow

6. **Asset Identification:** What must be protected?
7. **Damage Scenario:** What goes wrong if the asset is compromised?
8. **Threat Scenario:** What attack could cause the damage?
9. **Attack Path Analysis:** How could an attacker realistically reach and exploit the asset?
10. **Attack Feasibility:** How difficult is the attack? (access, knowledge, tools, time, opportunity)
11. **Impact Rating:** How severe is the damage? (safety, operational, privacy, financial, regulatory)
12. **Risk Value:** Combine feasibility and impact to produce a risk value.
13. **Risk Treatment:** Decide to avoid, reduce, transfer, or accept.
14. **Cybersecurity Goal:** State the required protection outcome and assign a CAL.
15. **Traceability:** Link goal through requirements to test evidence.

6.2 Asset Identification

- Identify vehicle functions, data, credentials, update paths, and service capabilities needing protection.
- **Assets include:** customer PII, vehicle location data, control logic, cryptographic keys, calibration data, safety-relevant states, firmware images, diagnostic session credentials.
- The same asset may exist in multiple places: ECU flash, cloud storage, workshop tool database, HSM — each location requires separate protection.
- Prioritize assets whose compromise could affect safety (ASIL-relevant functions), privacy (GDPR-regulated data), or operational continuity (fleet availability).

6.3 Damage Scenario Categories

Category	Examples
Safety (S)	Injury or death due to loss of braking, steering, propulsion, or misleading sensor output causing ADAS failure
Operational (O)	Vehicle downtime, loss of functionality, service disruption, fleet grounding, unauthorized immobilization
Privacy (P)	Exposure of location history, route data, driving behavior, biometric or personal user data in violation of GDPR
Financial (F)	Warranty cost, recall campaigns, fleet service burden, billing fraud, regulatory fines (up to €35M under EU CRA)
Regulatory (R)	Approval delays, market access issues, R155 non-compliance findings, certificate revocation

6.4 Attack Feasibility Factors

Factor	Levels	Examples
Required Access	Remote / Local wireless / Physical / Supply-chain proximity	Internet-reachable API vs. OBD port vs. factory programming
Required Knowledge	Publicly available / Specialized vehicle knowledge / OEM-insider access	Published CVE vs. proprietary protocol vs. signing key access
Required Tools	Standard IT tools / Specialized automotive hardware / Custom exploit tooling	Burp Suite vs. CANoe vs. custom RF hardware
Time to Exploit	Hours / Days / Months of sustained effort	Known CVE with PoC vs. novel zero-day development
Window of Opportunity	Always-on (e.g., cellular) vs. service-mode only (e.g., OBD connector)	TCU exposed continuously vs. diagnostic session only when at dealer

6.5 Risk Matrix

Feasibility ↓ / Impact →	Low Impact	Medium Impact	High Impact	Critical Impact
Low Feasibility	QM – Monitor	Treat if low cost	Monitor + segment	Escalate if safety-linked
Medium Feasibility	Treat as planned	Treat now	High priority	High priority
High Feasibility	Treat now	Treat now	Critical	Critical
Exploit Active	Immediate	Immediate	Emergency response	Emergency response

6.6 Risk Treatment Options

- **Avoid:** Remove the risky feature, interface, or trust relationship entirely. Strongest treatment; not always feasible.
- **Reduce:** Add authentication, segmentation, monitoring, or secure coding controls. Most common treatment path.
- **Transfer:** Use contractual or service-level controls for some operational responsibilities (e.g., cloud SOC monitoring via a third party).
- **Accept:** Only after documenting why the residual risk is tolerable and after formal management review with evidence. Never the first option.

6.7 TARA Algorithm (Pseudocode)

```
function TARA(assets, threat_catalog, attack_paths):  
    results = []  
    for asset in assets:  
        damage = assess_damage_scenarios(asset) # S, O, P, F, R categories
```

```

for threat in find_threats(asset, threat_catalog):
    for path in find_paths(asset, attack_paths):
        feasibility = score_feasibility(path, threat) # 1-4 scale
        impact      = score_impact(damage, threat)   # 1-4 scale
        risk_value  = combine(feasibility, impact)
        treatment   = decide_treatment(risk_value)
        results.append({
            asset, threat, path,
            feasibility, impact, risk_value,
            treatment, cal_level,
            owner, due_date
        })
results.sort(key=lambda r: r.risk_value, reverse=True)
return results # Prioritized work list for cybersecurity goals

```

6.8 Cybersecurity Goals and CAL

A cybersecurity goal states the desired protection outcome for a system, domain, or interface. ISO/SAE 21434 assigns a Cybersecurity Assurance Level (CAL 1 to CAL 4) based on the risk value, determining the rigor of development and testing required.

CAL	Risk Level	Development Rigor	Test Depth
CAL 1	Low risk	Standard secure coding, basic security design	Basic functional and security testing
CAL 2	Moderate	Enhanced review and analysis, security architecture review	Security-focused integration tests, boundary testing
CAL 3	High	Formal security analysis required, adversarial review	Adversarial testing, fuzzing, attack simulation
CAL 4	Very high / safety-critical	Most rigorous methods applicable, independent review	Full pen test, independent security review, red team

Chapter 7: HARA and Safety-Security Integration

Hazard Analysis and Risk Assessment (HARA), defined in ISO 26262, is the functional safety equivalent of TARA. While TARA focuses on intentional threats, HARA focuses on hazardous events arising from system failures. In modern vehicles, cyber attacks can trigger the same hazardous events that HARA addresses. Safety and security engineering must therefore be coordinated.

7.1 HARA Overview (ISO 26262)

HARA identifies hazardous events by combining an operational situation with a failure mode of a safety-relevant system. For each hazardous event, three parameters are assessed to determine the Automotive Safety Integrity Level (ASIL):

- **Severity (S):** Potential injury or harm to persons exposed to the hazard.
- **Exposure (E):** Probability that the operational situation occurs.
- **Controllability (C):** Ability of persons exposed to the hazard to avoid it.

7.2 HARA Parameter Definitions

Parameter	Level	Description
Severity	S0	No injuries to persons
	S1	Light to moderate injuries, typically survivable
	S2	Severe to life-threatening injuries, survival probable
	S3	Life-threatening or fatal injuries; survival uncertain
Exposure	E0	Incredible — operating situation essentially never occurs
	E1	Very low probability
	E2	Low probability
	E3	Medium probability
	E4	High probability — occurs regularly in normal use
Controllability	C0	Controllable in general — most persons can avoid harm
	C1	Simply controllable — most persons can avoid harm with simple action
	C2	Normally controllable — most persons can avoid harm, but not trivially
	C3	Difficult or uncontrollable — 50% or fewer persons could avoid harm

7.3 ASIL Determination Matrix (Complete)

ASIL is determined by the combination of S, E, and C. ASIL D is the highest integrity level; QM (Quality Management) means no specific safety requirement beyond standard quality processes applies.

S / E	C0	C1	C2	C3
S1 / E1	QM	QM	QM	QM

S / E	C0	C1	C2	C3
S1 / E2	QM	QM	QM	ASIL A
S1 / E3	QM	QM	ASIL A	ASIL B
S1 / E4	QM	ASIL A	ASIL B	ASIL C
S2 / E1	QM	QM	QM	ASIL A
S2 / E2	QM	QM	ASIL A	ASIL B
S2 / E3	QM	ASIL A	ASIL B	ASIL C
S2 / E4	ASIL A	ASIL B	ASIL C	ASIL D
S3 / E1	QM	QM	ASIL A	ASIL B
S3 / E2	QM	ASIL A	ASIL B	ASIL C
S3 / E3	ASIL A	ASIL B	ASIL C	ASIL D
S3 / E4	ASIL B	ASIL C	ASIL D	ASIL D

7.4 Cyber-Triggered Hazardous Events

Cyber Scenario	Attack	Safety Hazard	S / E / C	ASIL	Security Control
Remote compromise of braking ECU — unauthorized deceleration		Loss of braking	S3/E4/C3	ASIL D	Secure boot + CAN MAC + gateway segmentation
Spoofing of electric park brake command		Unintended vehicle movement	S2/E3/C2	ASIL C	Command authentication + freshness counter
DoS attack on ADAS processing ECU		Loss of collision warning / emergency brake	S2/E3/C2	ASIL B	Rate limiting + watchdog + fallback mode
Spoofing instrument cluster display — wrong speed shown		Driver misinformation	S1/E4/C1	ASIL A	Message integrity check + IDS alert
Ransomware on infotainment cluster — black screen		No display alerts visible to driver	S1/E3/C1	QM	Secure boot + OS hardening
Manipulation of ADAS perception via adversarial patch on road sign		Incorrect ADAS decision (SOTIF-linked)	S3/E2/C3	ASIL C	Adversarial robustness testing + SOTIF validation + CAL 3
Tampering with battery management system		Thermal runaway / fire	S3/E2/C3	ASIL D	Secure boot + dedicated BMS HSM + safety monitor

7.5 HARA-TARA Integration Pattern

- Safety engineers identify hazardous events and safety goals via HARA, producing ASIL assignments and safe states.

- Cybersecurity engineers identify assets and threats via TARA. For any asset that is safety-critical (ASIL A–D), the corresponding CAL requirement is elevated.
- **Shared work products:** The safety-critical function list from HARA feeds the asset identification step of TARA.
- Cyber threats that could compromise ASIL-D functions must be treated with CAL 4 rigor (most stringent, independent review required).
- Degraded modes and safe states defined in HARA must themselves be secured against cyber manipulation — an attacker should not be able to force a safe state or prevent exit from it.
- **SOTIF integration (ISO 21448:2022):** For ADAS functions, TARA scope must include adversarial inputs that could trigger SOTIF-relevant failures, coordinating with the SOTIF analysis team.

I KEY PRINCIPLE

A cyber attack that compromises a safety mechanism with ASIL D requirements must be treated with the same engineering rigor as an ASIL D random hardware failure. Security of safety functions is not optional.

Chapter 8: Secure Architecture Principles

The best security controls are built into the architecture, not patched on under release pressure. These six design habits prevent most avoidable security issues. The architecture should make insecure behavior hard to express, not just hard to detect.

#	Principle	Engineering Meaning
1	Least Privilege	Give each component only the permissions it needs. An infotainment ECU should not be able to write to powertrain CAN. A cloud service should only access the vehicle data it needs for its stated function.
2	Defense in Depth	Use layered controls so one failure is not catastrophic. Combine gateway filtering, ECU-level MAC, IDS, and backend anomaly detection.
3	Segmentation	Separate safety-critical, infotainment, and external networks. ADAS and body domains must not share bus segments. Ethernet VLAN boundaries must be enforced by the gateway.
4	Fail Securely	A fault should degrade to a safe and known state, not silently become dangerous or expose a bypass path. Safe states must themselves be secured.
5	Cryptographic Trust	Sign firmware, authenticate endpoints, and protect keys in hardware. Never use shared fleet keys. Each vehicle must have a unique cryptographic identity.
6	Traceability	Link security requirements to design decisions, test results, and release evidence. Untraceable controls cannot be audited.

8.1 Defense in Depth Layers

Layer	Focus	Typical Controls
Physical	Tamper resistance	Secure harnesses, debug port control, HSM enclosure, EMC shielding, counterfeit ECU detection
Network	Traffic control	Segmentation, VLAN, gateway filtering, MAC/CMAC, rate limiting, IDS, firewall rules
Compute	Code protection	Memory isolation, ASLR, MPU, stack canaries, secure boot, least privilege OS, container isolation
Application	Input and logic	Input validation, authorization checks, fail-safe logic, audit logging, API access control
Operations	Lifecycle management	Monitoring, patching, incident response, key rotation, revocation, SBOM CVE tracking

8.2 Secure Gateway Design

The gateway is the most important single security control in the vehicle. A well-designed gateway:

16. **Classifies** each message: type, sender domain, and applicable security policy.
17. **Validates** source identity, message freshness (freshness counter / MAC), and allowed routing.
18. **Filters** — blocks unsupported, unsafe, or rate-abusive traffic before forwarding.
19. **Forwards** only approved traffic to the destination domain.
20. **Observes** — records anomalies for alerting and forensic analysis.
21. **Isolates** — can segment a compromised domain without disrupting the rest of the vehicle.

8.3 Key Management Checklist

- Define who can generate, store, rotate, recover, and revoke each key type.
- Use unique keys or identities per vehicle — never share cryptographic secrets across a fleet.
- Protect signing keys in offline or HSM-backed environments with role-separated access.
- Validate provisioning: every ECU must receive exactly the identity and privileges intended.
- Plan for key compromise response: revocation paths must exist before a breach occurs.
- Implement key lifecycle: generation → provisioning → operational use → rotation → revocation → archival.

8.4 AUTOSAR SecOC — Secure Onboard Communication

AUTOSAR Secure Onboard Communication (SecOC) provides a standardized framework for authenticated inter-ECU communication over CAN, CAN-FD, and Automotive Ethernet:

- SecOC appends a Message Authentication Code (MAC, typically AES-128-CMAC) and a freshness value (counter) to PDUs.
- The receiving ECU verifies the MAC before processing the message — preventing injection and replay.
- **AUTOSAR R24-11 (November 2024)** introduced: AES-256-GCM support for Ethernet PDUs, CMAC-128 with 24-bit truncated MAC option for CAN bandwidth optimization, and enhanced freshness manager specifications.
- Key distribution for SecOC requires a secure key management infrastructure — typically HSM-based, provisioned at manufacturing.
- SecOC is mandatory for ASIL B and above safety-critical CAN communication in most OEM security specifications.

Chapter 9: Secure Boot, Cryptography, and Identity

Boot-time verification is the gatekeeper for everything that runs later. Cryptographic trust anchors the chain from hardware root of trust through to vehicle function. This chapter covers secure boot, HSMs, PKI, key lifecycle management, and 2024-updated post-quantum cryptography standards.

9.1 Secure Boot Chain

Stage	Component	Function	Security Property
Stage 0	ROM Bootloader	Immutable root of trust in hardware (OTP/ROM/HSM)	Hardware-rooted; cannot be overwritten
Stage 1	Stage 1 Bootloader	Verifies next stage using root key	Signature verification; anti-rollback check
Stage 2	OS / RTOS	Checks integrity and policy	Signed and measured image
Stage 3	Application	Allowed to operate only after chain validates	Trusted execution only after chain passes
Attestation	TPM / HSM	Records measurements for remote verification	PASS / HALT on verification failure

9.2 Secure Boot Pseudocode

```
function boot():
    root_key = read_hardware_root_of_trust() # Immutable in ROM/OTP/HSM
    stagel   = load_image("stagel_bootloader")

    if not verify_signature(stagel, root_key):
        halt_boot("invalid signature - stagel") # HALT immediately

    if not anti_rollback_check(stagel.version, min_allowed_version):
        halt_boot("rollback attempt blocked")

    measurement = hash_sha256(stagel)
    extend_trusted_measurement_register(measurement) # For remote attestation

    # Repeat for each subsequent stage (OS, application)
    handoff(stagel) # Each stage verifies the next before handoff
```

KEY PRINCIPLE

Root-of-trust keys and boot measurements must be stored in hardware (OTP fuses or HSM). Rollback protection prevents attackers loading old vulnerable firmware after a patch. Recovery mode must itself be secure — a compromised recovery path defeats the entire chain.

9.3 Hardware Security Modules (HSM)

- Hardware-backed storage for private keys, seed material, and critical cryptographic operations — software cannot extract the raw key material.
- HSM functions: secure boot measurement, message authentication (MAC/CMAC), key derivation (KDF), asymmetric operations (ECDSA sign/verify), and secure storage.
- Separation between security processor and application processor limits blast radius of a software compromise — even a full OS compromise cannot extract HSM-protected keys.

- **AUTOSAR R24-11 (November 2024):** Updated Crypto Service Manager (CSM) API with support for AES-256-GCM, ML-KEM key encapsulation, and enhanced HSM provider interface.
- Secure provisioning, key injection, and lifecycle management are equally critical as the HSM hardware itself — a perfectly designed HSM can be compromised by poor provisioning.

9.4 Public Key Infrastructure (PKI) for Vehicles

- PKI provides devices and services a way to prove identity cryptographically at scale.
- Vehicle certificates are issued at manufacturing, covering ECU identity, update authorization, and V2X pseudonym rotation.
- Certificate lifecycle: issuance → storage → operational use → renewal → revocation → recovery — all must be designed before production.
- Role separation between Registration Authority (RA) and Certificate Authority (CA) is essential for audit trail and breach containment.
- **Key compromise response time** is more critical than key algorithm choice: plan revocation infrastructure before it is needed.
- V2X pseudonym certificates (IEEE 1609.2 / ETSI TS 102 941) provide location privacy via certificate rotation at defined intervals.

9.5 Cryptographic Algorithm Selection (June 2026)

⚠ NOTE

Post-Quantum Cryptography: NIST finalized FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) in August 2024. These are the authoritative replacements for RSA and classical ECC for quantum-resistant applications. Migration planning should begin now for systems with >10-year service life.

Function	Recommended Algorithm	Key / Output Length	Notes
Symmetric encryption	AES-GCM	128 or 256-bit	GCM provides confidentiality + integrity (AEAD)
Message authentication	AES-CMAC / HMAC-SHA-256	128-bit MAC / 256-bit	CMAC mandatory for AUTOSAR SecOC; HMAC for IP-based
Digital signatures	ECDSA (P-256 or P-384)	256 or 384-bit EC	Prefer over RSA for embedded; lower footprint
Key exchange	ECDHE P-256 or higher	P-256 or P-384	Forward secrecy for TLS sessions
Hashing	SHA-256 / SHA-384	256 or 384-bit output	SHA-1 / MD5 must NOT be used; SHA-3 acceptable
PQC Key Encapsulation	ML-KEM (FIPS 203)	ML-KEM-768 / ML-KEM-1024	NIST final standard Aug 2024; replaces CRYSTALS-Kyber
PQC Digital Signatures	ML-DSA (FIPS 204)	ML-DSA-44 / ML-DSA-65 / ML-DSA-87	NIST final standard Aug 2024; replaces CRYSTALS-Dilithium

Function	Recommended Algorithm	Key / Output Length	Notes
PQC Stateless Hash Sigs	SLH-DSA (FIPS 205)	SLH-DSA-SHA2-128f / 256s	NIST final standard Aug 2024; replaces SPHINCS+
TLS (vehicle ↔ cloud)	TLS 1.3	Per cipher suite	TLS 1.2 acceptable short-term; TLS 1.0/1.1 must be disabled

PQC Migration Timeline Recommendation

- **2024–2026:** Inventory all cryptographic assets; identify systems with >10-year service life; begin hybrid classical+PQC experiments.
- **2026–2028:** Deploy PQC in new platforms for key exchange (ML-KEM); maintain classical fallback for interoperability.
- **2028–2030:** Extend PQC to signatures (ML-DSA) for firmware signing and V2X certificates in new vehicle programmes.
- **2030+:** Full PQC migration for all safety-critical cryptographic operations; classical-only systems require risk acceptance.

Chapter 10: Secure OTA Update Pipeline

Software updates are both a security asset (fixing vulnerabilities) and a security risk (compromised updates). They must be authenticated, traceable, and reversible. This chapter covers the full Build-to-Recover pipeline and UNECE R156 requirements.

10.1 OTA Pipeline Stages

#	Stage	Activities	Security Control
1	Build	Code review, static analysis, dependency check, SBOM generation	Signed artifact, SBOM, locked dependencies, SLSA provenance
2	Package	Create manifest with version, hash, target ECU, rollout scope	Manifest integrity, HSM-backed signing ceremony, witness log
3	Distribute	Deliver via encrypted channel, staged rollout by VIN cohort	TLS 1.3 / HTTPS, authorization check, rate control
4	Install	Verify integrity, write to inactive A/B slot	Hash verification, signature check, slot isolation
5	Activate	Post-install self-test, ECU health check, boot counter check	Watchdog timer, boot counter, ECU self-test pass/fail
6	Recover	Automatic rollback on failure, safe mode activation	Known-good fallback image, rollback policy, vendor escalation path

10.2 OTA Algorithm (Pseudocode)

```
function ota_update(package, vehicle_context):
    if not verify_manifest_signature(package.manifest):
        reject("manifest signature invalid")

    if not verify_package_signature(package):
        reject("package signature invalid")           # HSM-verified

    if not policy_allows(vehicle_context.vin, package.version):
        reject("vehicle not in rollout scope or version not permitted")

    if not compatible(package.target_ecu, vehicle_context.ecu_config):
        reject("compatibility check failed")

    if not safe_install_window(vehicle_context):      # Speed, battery, network
        defer("install conditions not met - retry later")

    backup_current_image(package.target_ecu)
    install_to_inactive_slot(package)

    if health_check_passes(package.target_ecu):
        activate_new_slot()
        record_update_result(SUCCESS, vin, timestamp, backend_id)
    else:
        rollback_to_backup()
        record_update_result(FAILURE, reason, vin, timestamp)
```

10.3 Update Controls Checklist

- **Signed packages** with signed manifest (version, target ECU, hash, rollout scope, expiry).
- **Anti-rollback policy:** Vehicles must not accept older, potentially vulnerable firmware — enforce minimum version counters in HSM-protected OTP storage.
- **Secure fallback image** for recovery from failed updates — the fallback image itself must be signed and integrity-protected.
- **A/B partitioning:** Never write to the active slot; activate only after passing post-install health checks.
- **Audit logging:** Who authorized the update, when, from which backend, for which VIN range, and what the install result was.
- **Customer communication plan:** Update release notes, vehicle UI indicators, consent (where required by regulation), and support escalation path.
- **UNECE R156 evidence:** Update campaign records, compatibility verification evidence, anti-rollback evidence, and customer communication records.

10.4 SBOM and Software Supply Chain

- A Software Bill of Materials (SBOM) in **CycloneDX** or **SPDX** format inventories components, versions, licenses, and origins for every release artifact.
- SBOM enables rapid CVE impact assessment when a new vulnerability is published — which VINs, which ECUs are affected?
- Release artifacts should include provenance, build metadata (commit hash, build timestamp, toolchain version), and signing evidence.
- **SLSA (Supply-chain Levels for Software Artifacts) Framework:** SLSA Level 3+ requires hermetic builds, locked toolchains, and build provenance attestation — automotive-applicable for safety-critical software components.
- Supplier deliverables must include SBOM, version declarations, and security attestations per ISO/SAE 21434 Clause 7.

10.5 Build Pipeline Security Controls

Pipeline Stage	Controls
Source Control	Protected branches, mandatory peer review, GPG/SSH commit signing, role-based write access, branch protection rules
Build Integrity	Reproducible builds (hermetic environment), locked dependency versions, isolated build runners, SLSA Level 3+ provenance
Secrets Management	No hard-coded keys in code or config; vault-managed credentials with rotation; audit log for secret access
Release Signing	HSM-backed signing key, protected signing ceremony, dual-control authorization, audit log of all signing operations
Release Gates	Test completion gate, vulnerability scan (CVSS $\geq$ 9.0 blocks release), dependency review, management approval record
Post-Release Monitoring	Rollout telemetry, anomaly alerts on update failures, fleet health dashboard with VIN-level install status

Chapter 11: Secure Development and Security Testing

Security controls need proof. A good program uses layered testing that moves from fast static checks to adversarial penetration testing. Each test type has specific strengths and coverage targets. This chapter covers both development hygiene and the verification pyramid.

11.1 Secure Coding Essentials

- Validate all external input, even when it originates from other ECUs or trusted services — do not assume internal trust.
- Prefer explicit state machines and narrow interfaces over loosely coupled command handlers.
- Treat memory safety (buffer overflows, use-after-free, integer overflow, format string vulnerabilities) as first-order security concerns — they account for >70% of exploitable automotive vulnerabilities.
- Disable unused ports, services, and debug features (JTAG, UART, SWD) in production images. A feature disabled at boot is safer than one blocked by policy alone.
- Parse all structured binary data defensively: check lengths, canonical encoding, and boundary conditions before dereferencing any pointer.
- **MISRA C:2023** and **AUTOSAR C++14** coding guidelines provide automotive-specific rules addressing security and safety concerns.
- **AUTOSAR SecOC** provides a standardized framework for authenticated inter-ECU communication over CAN and Ethernet with MAC and freshness counters.

11.2 Authentication and Authorization

Concept	Question Answered	Applies To	Common Controls
Authentication	Who is the caller?	Diagnostics, cloud APIs, workshop tools, update channels, inter-ECU service requests	PKI certificates, session tokens, challenge-response, mutual TLS
Authorization	What is the authenticated caller allowed to do?	All authenticated sessions — must be least-privilege and state-aware	Role-based access, service permissions, session state machine, resource ownership check

i KEY PRINCIPLE

Both are required. A system that authenticates without checking authorization grants attackers full access once they forge any identity. A system that authorizes without authenticating can be trivially bypassed by spoofing.

11.3 Security Test Pyramid

#	Method	Strength	Best Used For
1	Static Analysis (SAST)	Fast, early, broad coverage	Unsafe patterns, coding rule violations, taint analysis, CWE detection
2	Unit Tests	Precise, cheap, fast regression	Algorithms, parsers, crypto wrappers, state machine logic
3	Integration Tests	Cross-component truth	ECU-to-ECU flows, service interactions, gateway filtering rules
4	Fuzzing	Adversarial input	Protocol parsers, service handlers, OTA

#	Method	Strength	Best Used For
		discovery	decoders, CAN frame processing
5	HIL / SIL	System realism, regression safety	Vehicle-level behavior, update campaigns, domain interaction
6	Penetration Testing	Real attacker chaining	Complex exploitation paths, gateway bypass, backend abuse, supply chain attack simulation

11.4 Fuzzing Harness (Pseudocode)

```
function fuzz_target(raw_input):
    # Shape input to match protocol constraints
    msg = mutate_to_protocol_shape(raw_input)
    try:
        result = parse_and_handle_message(msg)
        if result.state_corrupted:
            report_finding("state corruption", raw_input)
        validate_state_transitions(result)
    except (SegFault, BufferOverflow, Timeout):
        report_finding("crash/timeout - candidate security bug", raw_input)
        save_reproducer(raw_input)

    # Coverage-guided: update corpus if new code paths found
    update_coverage_corpus(raw_input)

# Automotive-specific fuzz targets:
# - UDS message parser (ISO 14229)
# - DoIP header decoder (ISO 13400)
# - SOME/IP service handler
# - OTA package manifest decoder
# - CAN frame DLC/payload handler
```

11.5 Penetration Testing

- Pen testing chains weaknesses the way a real attacker would — not just individual CVEs in isolation.
- Target objectives should be explicit: remote compromise, gateway pivoting, update abuse, data exfiltration, or backend API attack.
- Combine network (remote cellular/V2X), physical (OBD, debug ports, hardware teardown), service tool, and backend cloud test perspectives.
- Every critical finding must be triaged before release with documented risk acceptance or remediation evidence.
- **Scope example for a new vehicle platform:** cellular-to-vehicle attack path, Bluetooth pairing weaknesses, OBD session escalation, OTA package replay, cloud API BOLA testing.

11.6 Release Readiness Gate

- All CAL-assigned security requirements must have test coverage with traceable pass results.
- Signed binaries, update manifests, and rollback plans must match the approved release content.
- Monitoring, IDS configuration, and incident response playbooks must be operationally ready before field deployment.
- SBOM must be complete, current to the release, and available for rapid CVE impact assessment.

i KEY PRINCIPLE

If the team cannot explain the security state at release, the release is probably not ready.

PART III

FIELD OPERATIONS

Detection, monitoring, incident response, and operational security at scale.

Chapter 12: Detection, Logging, and Anomaly Monitoring

Chapter 13: Incident Response Playbook

Chapter 12: Detection, Logging, and Anomaly Monitoring

Security does not stop at launch. The field fleet is where the most important real-world data appears. The goal is to detect abuse early, understand it quickly, and contain it before it causes harm or spreads across the fleet.

12.1 Monitoring Pipeline

Stage	Activity
Signals	Collect: bus traffic, login events, update activity, sensor sanity, ECU health, network connections
Normalize	Convert raw events to a common structured event format (timestamp, source, type, severity, payload)
Score	Apply rules, thresholds, behavioral baselines, and correlation across ECUs and time windows
Alert	Create triage-worthy security events with severity, context, confidence score, and ASIL relevance
Respond	Contain the issue, preserve evidence, notify relevant teams and systems, execute playbook

12.2 IDS Algorithm (Pseudocode)

```
function detect(event_stream, baseline_model):
    alerts = []
    for event in event_stream:
        # Rule-based detection (known bad patterns) - fast path
        if matches_known_rule(event, rule_set):
            alerts.append(create_alert(event, type=RULE, severity=HIGH))

        # Anomaly-based detection (deviation from baseline) - slower path
        deviation = compute_deviation_score(event, baseline_model)
        if deviation > THRESHOLD:
            alerts.append(create_alert(event, type=ANOMALY, score=deviation))

    # Correlate across ECUs and time window
    alerts = deduplicate_and_correlate(alerts)
    alerts.sort(key=lambda a: safety_relevance(a), reverse=True)
    return alerts
```

12.3 Example Detection Rules

```
# Reboot storm detection
if ecu_reboots(target=ECU_ID, window=60s)>REBOOT_THRESHOLD:
    if not vehicle.in_maintenance_mode:
        raise_alert("reboot storm - potential DoS or firmware issue", severity=HIGH)

# Unknown diagnostic tool
if diagnostic_session_source not in approved_tool_identifiers:
    block_session()
    log_event("unauthorized diagnostic session attempt", source_ip, vin)

# CAN message rate anomaly (flooding or injection)
if message_rate(CAN_ID, window=1s)>baseline_rate * 3:
```

```

isolate_bus_segment()
raise_alert("CAN message rate anomaly - possible injection or flooding")

# OTA update from unauthorized server
if update_source_certificate not in trusted_anchor_store:
    reject_update()
    raise_alert("OTA from untrusted source", cert_hash, vin)

# Failed authentication burst (brute force)
if failed_auth_count(source, window=300s)>5:
    block_source(source, duration=3600s)
    raise_alert("authentication brute force detected", source, vin)

```

12.4 Logging Requirements

- Logs must be time-stamped (synchronized to a secure time source — GPS or NTP with integrity), tamper-evident, and integrity-protected (HMAC chain).
- **Security-relevant events to log:** failed authentication, successful privileged sessions, firmware changes, key operations, diagnostic programming, anomaly triggers, update install/rollback events.
- Protect logs from deletion, replay, and over-collection of personal data — GDPR/privacy compliance requires purpose limitation on log retention.
- Selected security telemetry should be forwarded to backend SOC for fleet-level correlation across VINs.
- Good logs are usable for forensic investigation, compliance audit, and root-cause analysis — three different but overlapping use cases.

12.5 Rule-Based vs Anomaly-Based IDS

Aspect	Rule-Based	Anomaly-Based
Speed	Very fast — $O(1)$ lookup	Higher latency — model scoring required
Explainability	High — rule is directly readable	Lower — requires model interpretation
Coverage	Only known threats / signatures	Can detect novel or stealthy patterns
False positive rate	Low if rules are well tuned	Higher; needs careful calibration and fleet-baseline training
Best practice	Use for high-confidence, fast block decisions	Use for fleet-scale outlier and behavioral analysis

i KEY PRINCIPLE

Use a hybrid approach: rule-based detection for immediate known-bad blocking, anomaly detection for fleet-scale behavioral analysis. Both must be versioned, tested, linked to response playbooks, and updated when the vehicle configuration changes.

Chapter 13: Incident Response Playbook

Prepared teams recover faster, preserve evidence better, and avoid panicked decisions under pressure. Incident response must be planned and practiced before the first real incident occurs. The cardinal rule during any automotive security event: safety first, evidence second, speed third.

13.1 Six-Phase Incident Response Lifecycle

#	Phase	Key Actions	Evidence to Preserve
1	Prepare	Define roles, escalation paths, tools, and drills. Establish CSIRT and OEM-VSOC contacts. Register with Auto-ISAC.	Playbook, contact list, authorized tools list, drill records
2	Detect	Validate the alert, determine scope and severity. Confirm true positive via IDS and vehicle telemetry.	Alert log, IDS output, initial telemetry snapshot
3	Contain	Segment affected ECUs or vehicles, revoke active sessions, disable implicated services. Prevent propagation.	Network isolation records, session revocation timestamps
4	Eradicate	Remove root cause: patch, reflash, revoke credentials, eliminate backdoors.	Before/after firmware hashes, patch records, revocation evidence
5	Recover	Restore service: verified reflash, certificate reissuance, functional test, safety validation.	Post-recovery test results, vehicle telemetry, safety sign-off
6	Learn	Root cause analysis, TARA update, design and process improvements, supplier feedback, Auto-ISAC sharing.	Postmortem report, TARA revision, lessons register

13.2 Forensics Workflow

22. **Preserve:** Collect logs, firmware images, network captures, timestamps — without modifying evidence (read-only image of affected ECU storage).
23. **Correlate:** Link vehicle-side, cloud backend, service tool, and update system records into a unified timeline.
24. **Analyze:** Identify root cause, complete attack chain, extent of compromise, and affected VIN population.
25. **Report:** Produce clear findings, affected VIN scope, corrective actions with owners and due dates, and regulatory notification assessment (EU CRA 24-hour reporting obligation).

13.3 Vulnerability Management Lifecycle

Activity	Description
Inventory	SBOM-based CVE mapping to affected software versions and vehicle configurations — run continuously against NVD/OSV feeds
Triage	Score with context (exploitability in-vehicle, safety relevance, fleet exposure) — not raw CVSS alone; use EPSS + ASIL relevance weighting

Activity	Description
Remediation	Track fix through development → test → OTA campaign → fleet confirmation with VIN-level install tracking
Communication	Align customer, service, legal, compliance, and engineering workflows; assess EU CRA 24h reporting obligation
Closure	Confirm fix deployed to affected VIN scope; record residual risk where full remediation is not possible

13.4 Auto-ISAC and Information Sharing

- **Automotive Information Sharing and Analysis Center (Auto-ISAC):** Industry body for sharing automotive cybersecurity threat intelligence. Members include major OEMs, Tier-1 suppliers, and technology providers.
- Membership enables access to anonymized threat intelligence, attack indicators, and vulnerability advisories specific to automotive systems.
- Participating OEMs can reduce mean time to detect (MTTD) for novel attack patterns by benefiting from collective fleet intelligence.
- Information sharing is governed by Traffic Light Protocol (TLP) — from TLP:WHITE (public) to TLP:RED (named recipients only).
- NHTSA 2024 guidelines explicitly encourage Auto-ISAC participation as a best practice for US market compliance readiness.

13.5 Operations Dashboard — Decision-Oriented Metrics

- **Alerts:** Authentication failures by ECU, update anomalies, integrity drift events, rate threshold crossings.
- **Exposure:** Open CVEs by ASIL relevance, active exploit campaigns in Auto-ISAC feeds, affected VIN count.
- **Response:** Containment status, reflash progress by VIN, certificate revocation completion rate.
- **Quality:** False positive rate (must be tunable), detection coverage, **Mean Time to Detect (MTTD)**, **Mean Time to Recover (MTTR)** — both tracked per severity tier.
- **Fleet:** Regional distribution, usage profile, rolling service window progress.
- **Learning:** Top root causes this quarter, preventive actions in progress, design changes triggered.

PART IV

EMERGING DOMAINS

V2X, EV charging, ADAS/SOTIF, cloud security, AI/ML, and privacy.

Chapter 14: Special Domains&Cross-Cutting Concerns

Chapter 14: Special Domains&Cross-Cutting Concerns

Modern vehicles are connected, electrified, automated, and distributed across cloud and supplier ecosystems. Security scope must stretch beyond the vehicle body. This chapter addresses the domains requiring special attention in 2026 and beyond.

14.1 V2X Security

- V2X (Vehicle-to-Everything) messages must be authenticated and protected against replay, injection, and misbehavior.
- **IEEE 1609.2a (2024 Amendment)**: Updates to the V2X certificate framework including revised certificate structure for application-level authorization, enhanced revocation mechanisms, and improved pseudonym certificate linkage value handling.
- Certificate and pseudonym management are central to trust and location privacy — pseudonym certificates rotate regularly to prevent tracking.
- Vehicles must not blindly trust a single external V2X message even if cryptographically valid — misbehavior detection is required.
- **ETSI TS 102 940 / TS 103 097** define the PKI and certificate framework used in European V2X deployments; **IEEE 1609.2** is used in North America.
- Misbehavior detection and revocation handling are key operational concerns for V2X PKI authorities — a vehicle sending false safety messages must be revocable quickly.

14.2 EV Charging and Energy Interfaces

- **ISO 15118-2 (2014)** defines the core encrypted, authenticated communication between vehicle and charging station (Plug & Charge — PnC protocol).
- **ISO 15118-20 (2022)** extends the protocol to support AC charging, wireless power transfer (WPT), and bidirectional power flow (V2G) with enhanced authentication mechanisms and improved certificate handling.
- **Plug&Charge (PnC)**: The vehicle presents a contract certificate to the charging station, enabling automatic authentication, authorization, and billing without user interaction — requires a complete V2G PKI ecosystem.
- A compromised charger can become a data exfiltration gateway or a vehicle network entry point via the ISO 15118 communication channel.
- Fleet charging and home charging create different risk ownership boundaries — fleet operators control the charging infrastructure; consumer home charger security is outside the OEM's direct control.
- Backend billing and energy service APIs need the same API security controls (BOLA prevention, OAuth 2.0, rate limiting) as cloud vehicle services.

14.3 ADAS and Automated Driving — SOTIF Intersection

- Sensor inputs, fusion pipelines, compute platforms, and actuation paths must be protected end-to-end.
- **SOTIF (ISO 21448:2022) and Cybersecurity**: Adversarial ML attacks can produce SOTIF-relevant failures via intentional external manipulation — distinct from random hardware faults and from software defects.
- The SOTIF process identifies triggering conditions and operational scenarios where the system functions correctly but produces unsafe outputs — adversarial attacks are a category of deliberate triggering condition.
- SAE Level 3+ systems increase reliance on software correctness and remove the driver from the safety loop — the security stakes increase proportionally.

- Fallback and driver handover behavior must themselves be secured against cyber manipulation — an attacker should not be able to force an unintended takeover request or prevent a legitimate one.
- Model updates and calibration data changes for ADAS functions need controlled SUMS-compliant deployment and re-validation cycles.

14.4 Cloud-Vehicle Security

- Backend APIs, identity systems, update platforms, and telemetry services can dominate the vehicle trust model.
- A cloud platform compromise can affect thousands of vehicles simultaneously — blast-radius planning and multi-region isolation are critical design requirements.
- Token handling, service segregation, rate limiting, and least-privilege API access (BOLA/BFLA prevention) are essential.
- Signing infrastructure must be in a dedicated, highly controlled environment separate from general cloud workloads — compromise of OEM signing keys is a catastrophic event.
- **Zero Trust Architecture (NIST SP 800-207)**: Treat every service-to-service API call as untrusted regardless of origin; enforce authorization at each boundary.

14.5 Privacy Engineering

- Vehicles collect location history, route patterns, driving style, in-cabin audio/video, contacts, and biometric data — GDPR classifies several of these as special category data.
- **Data minimization**: Collect only what is needed for the stated purpose; document purpose limitation in data protection impact assessments (DPIA).
- **Retention discipline**: Data deleted on schedule; long retention multiplies breach impact and regulatory exposure.
- **GDPR Article 25 (Privacy by Design)** requires privacy to be considered from the architecture design phase, not added as a post-release feature.
- Access control on logs and telemetry must enforce purpose limitation — a diagnostic engineer should not have access to passenger cabin audio recordings.

14.6 AI/ML in Vehicle Systems

Concern	Description	Mitigation
Model integrity	Verify deployed models have not been tampered with between training and inference	Sign model weights + config; verify signature at runtime before loading
Data drift	Production input distributions may diverge from training data, degrading model safety	Monitor inference anomalies; retrigger validation when distribution shift detected
Adversarial manipulation	Physical-world attacks (adversarial patches on road signs) can fool perception models	Adversarial robustness testing + ensemble defenses + SOTIF validation
Prompt injection (GenAI)	Third-party content can inject adversarial instructions into in-vehicle AI assistants	Input sanitization + sandboxed execution + output validation before action
Model update governance	ML model updates must follow the same SUMS process as traditional firmware	Signed model artifacts + SUMS-compliant OTA + re-validation cycle

14.7 Threat-Informed Control Selection

```
function choose_controls(threat_scenario):
    controls = []
    if threat_scenario.involves("spoofing"):
        controls += ["mutual authentication", "PKI certificates", "freshness counter"]
    if threat_scenario.involves("tampering"):
        controls += ["digital signature", "secure boot", "CMAC/AES-GCM"]
    if threat_scenario.involves("denial_of_service"):
        controls += ["rate limiting", "segmentation", "watchdog", "fallback mode"]
    if threat_scenario.involves("data_theft"):
        controls += ["AES-GCM encryption", "least privilege", "data minimization"]
    if threat_scenario.involves("elevation_of_privilege"):
        controls += ["authorization check", "session control", "segment isolation"]
    if threat_scenario.involves("adversarial_ml"):
        controls += ["adversarial robustness test", "SOTIF analysis", "model integrity
check"]
    return prioritize(controls, asset.ASIL_or_CAL)
```


PART V

REFERENCE MATERIAL

Checklists, maturity model, and quick-reference sheets for daily use.

Chapter 15: OEM and Supplier Readiness Checklist
Chapter 16: Maturity Roadmap and Common Mistakes
Chapter 17: Quick Reference Summary and Cheat Sheet

Chapter 15: OEM and Supplier Readiness Checklist

This checklist provides a compact set of questions for design reviews, supplier audits, and release gate assessments. Use it at SOP, SOP+2, and post-release programme gates. A "No" answer requires a documented risk acceptance or remediation plan.

Domain	Question	Evidence Expected
Architecture	Are trust boundaries, assets, interfaces, and security zones documented?	Architecture diagram, trust boundary map, asset register
Development	Are secure coding standards, secrets handling, and code review gates enforced?	Coding standard, SAST report, review records, CI gate evidence
Cryptography	Are key management, provisioning, and algorithm selection documented and verified?	Key management plan, HSM provisioning record, algorithm register
OTA	Are signing, integrity, rollback, and update authorization fully implemented and tested?	OTA test evidence, manifest signing log, rollback test evidence
PQC Readiness	Has a PQC migration roadmap been assessed for systems with >10-year service life?	Cryptographic asset inventory, PQC migration plan, risk acceptance for classical-only systems
Supplier	Are security requirements flowed down and evidenced by supplier deliverables?	Security interface agreements, SBOM, supplier TARA, supplier audit records
Verification	Have abuse cases, fuzzing, and penetration testing been completed?	Test plan, findings tracker, pen test report, fuzz harness evidence
Production	Are key injection, certificate provisioning, and secure manufacturing processes controlled?	Provisioning SOP, audit log, factory test records, HSM injection evidence
Operations	Are monitoring, IDS, patching, and incident response operational for field vehicles?	IDS configuration, SOC runbook, patch process doc, MTTR/MTTD baseline
CSMS	Is the organization-level CSMS audited and certified (R155 requirement)?	CSMS audit report, certification records, management review minutes
SUMS	Is the SUMS documented, tested, and aligned with R156 requirements?	SUMS process document, update campaign records, anti-rollback evidence
Privacy	Has a Data Protection Impact Assessment (DPIA) been completed for vehicle telemetry?	DPIA report, data minimization evidence, retention schedule
AI/ML (if applicable)	Are ML model updates governed by SUMS and model integrity verified at runtime?	Signed model artifacts, SUMS alignment evidence, adversarial test results

i KEY PRINCIPLE

Use this checklist at design reviews, supplier scorecards, and SOP/SOP+2 release gates. Map each line to the responsible owner and a due date. Unresolved items at SOP require explicit management

risk acceptance.

Chapter 16: Maturity Roadmap and Common Mistakes

Building automotive cybersecurity capability is a multi-year journey. This chapter provides a staged maturity model, an implementation sequence, and a list of recurring mistakes observed in real programmes.

16.1 Cybersecurity Maturity Model

Level	Name	Characteristics	Typical Gap
1	Ad Hoc	Heroics, manual checks, limited visibility, reactive patching only	No process; outcomes depend entirely on individuals
2	Repeatable	Basic process, some ISO 21434 alignment, improving discipline	Inconsistent execution; gaps in traceability and supplier management
3	Managed	Metrics, governance, evidence, cross-functional ownership, CSMS certified	Limited automation; reactive operations; partial IDS coverage
4	Optimized	Continuous improvement, automation, fleet learning, proactive threat intelligence	Scaling to growing fleet and SDV supply chain complexity
5	Resilient	Fast response, strong ecosystem trust, proactive design, industry information sharing	Maintaining edge as threat landscape and vehicle technology evolve simultaneously

16.2 Implementation Roadmap

26. **Establish:** Define scope, roles, and a common language across engineering and compliance teams.
27. **Inventory:** Create an asset and interface register for each platform or vehicle line.
28. **Analyze:** Introduce TARA, HARA cross-reference, SOTIF alignment, and requirements traceability into the design process.
29. **Secure:** Implement build pipeline security, OTA signing, gateway controls, SecOC, and provisioning process.
30. **Test:** Run static analysis, fuzzing, abuse case tests, and at least one pen test per platform before SOP.
31. **Monitor:** Deploy IDS and fleet monitoring before vehicle SOP — not after.
32. **Improve:** Measure using release evidence, vulnerability closure rate, MTTD/MTTR, and fleet telemetry.

16.3 Top Mistakes to Avoid

Mistake	Consequence
Late security	Treating security as a final checklist. Redesign costs are 10× greater post-architecture than at concept.
Weak identity	Shared fleet keys or missing auth on critical services. One key compromise affects every vehicle with that key.
No rollout control	Updating vehicles without compatibility, staging, and rollback. A bad update can brick thousands of vehicles simultaneously.

Mistake	Consequence
No observability	No IDS, logs, or fleet visibility. You cannot defend what you cannot see.
Single point of trust	One trust store, one gateway, or one backend with too much power. Compromise is catastrophic with no fallback.
Supplier blindspots	Assuming integrations are secure without SBOM, TARA, or security requirements evidence from suppliers.
Not tested recovery	Rollback and recovery paths are never tested under realistic failure conditions. They fail exactly when most needed.
Security theater	Documents that look compliant but donot reflect reality. Auditors and attackers both find the gap.
PQC deferred too long	Assuming quantum computers are a distant problem. Systems deployed today may still be operational when quantum attacks become practical.
No SOTIF coordination	Running TARA and SOTIF analysis in silos. Adversarial ML attacks that produce SOTIF failures require both CAL and ASIL treatment.

Chapter 17: Quick Reference Summary and Cheat Sheet

This chapter is a compact one-stop reference for engineers, managers, and reviewers who need fast recall of the most important concepts. Print it, pin it, reference it at design reviews.

17.1 Common Controls Cheat Sheet

Protect Against	Typical Controls
Identity spoofing	PKI certificates, hardware-backed keys, unique per-vehicle credentials, revocation infrastructure
Software tampering	Secure boot, signed firmware images, anti-rollback (version counter in OTP), hash verification
Network attacks	Segmentation, VLAN, gateway filtering, MAC/CMAC, rate limiting, IDS, firewall rules
Service misuse	Session authentication, authorization check (BOLA/BFLA prevention), rate limiting, audit logging
Field threats	IDS/monitoring, incident response playbook, patch campaigns, OTA rollback capability
Supply chain risk	SBOM (CycloneDX/SPDX), provenance records, code signing, CVE scanning, SLSA Level 3+
Key compromise	HSM storage, offline signing key, revocation infrastructure, rotation plan, dual-control signing
Privacy breach	Data minimization, AES-GCM at rest/transit, retention limits, access control, GDPR DPIA
Quantum threats	PQC migration plan, ML-KEM for key exchange, ML-DSA for signatures (FIPS 203/204/205)
Adversarial ML	Adversarial robustness testing, model integrity signing, SOTIF coordination, input validation

17.2 Standards Quick Reference

Standard	Scope	Key Requirement
ISO/SAE 21434:2021	Engineering method	TARA, cybersecurity goals, lifecycle work products, CAL 1–4
UNECE R155	Vehicle regulation	CSMS: governance, risk, monitoring, response, supplier management
UNECE R156	Update regulation	SUMS: signed updates, anti-rollback, compatibility, campaign evidence
ISO 26262:2018	Functional safety	HARA, ASIL A–D, safety goals, safety-critical software development
ISO 21448:2022 (SOTIF)	Intended functionality safety	Triggering conditions, out-of-distribution inputs, ADAS validation
NHTSA 2024	US best practice	Risk-based approach, SBOM, OTA security, EV charging, cloud API governance
EU CRA (2024)	Product security	Vulnerability handling, security updates for product lifetime; 24h ENISA reporting

Standard	Scope	Key Requirement
NIST FIPS 203/204/205	Post-quantum crypto	ML-KEM (key encapsulation), ML-DSA (signatures), SLH-DSA (hash sigs)
IEEE 1609.2a (2024)	V2X PKI	V2X certificate amendments, revocation, pseudonym management
ISO 15118-20 (2022)	EV charging security	Plug&Charge, V2G, WPT authentication; TLS 1.3 + ECC certificates

17.3 Three Memorable Rules

i KEY PRINCIPLE

Rule 1 — Design for trust boundaries. Every external interface is a potential entry point until it is explicitly controlled, authenticated, and authorized.

i KEY PRINCIPLE

Rule 2 — Verify every update and every identity. Unsigned code and unauthenticated commands are risks, not features. Sign everything, verify everything.

i KEY PRINCIPLE

Rule 3 — Assume failure and recover safely. Plan the rollback before the rollout. Test the recovery path as seriously as the happy path.

17.4 Best Learning Shortcut

⚠ NOTE

Attacker-mindset drill: For any feature or interface, ask: How would an attacker move from this entry point to causing the worst possible outcome? Then place control points along every step of that path. This single exercise surfaces more design gaps than any checklist.

17.5 Sources and References

- ISO/SAE 21434:2021 — Road vehicles — Cybersecurity engineering.
- ISO 26262:2018 — Road vehicles — Functional safety (Part 3: Concept phase, HARA).
- ISO 21448:2022 — Road vehicles — Safety of the intended functionality (SOTIF).
- UNECE UN Regulation No. 155 — Cybersecurity and cybersecurity management system.
- UNECE UN Regulation No. 156 — Software update and software update management system.
- NHTSA 2022/2024 — Vehicle Cybersecurity Best Practices (updated).
- ENISA 2022/2023 — Good Practices for Security of Smart Cars and Automotive Cybersecurity.
- EU Cyber Resilience Act (CRA) — Entered into force December 11, 2024.
- NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA) — Published August 2024.
- AUTOSAR SecOC R24-11 — Specification of Secure Onboard Communication.
- IEEE 1609.2a-2024 — Amendment to V2X Certificate Framework.
- ISO 15118-20:2022 — Road vehicles — Vehicle to grid communication interface.
- SAE J3061 — Cybersecurity Guidebook for Cyber-Physical Vehicle Systems (foundational reference).

- Auto-ISAC — Automotive Cybersecurity Best Practices (2023 edition).
- SLSA Framework (Supply-chain Levels for Software Artifacts) — slsa.dev.

PART APPENDICES

CASE STUDIES, TOOLS&REFERENCE

Real-world incidents, tool guides, maturity model, glossary, and PQC reference.

- Appendix A: Jeep Cherokee Remote Hack (2015)
- Appendix B: Tesla Security Program — Industry Benchmark
- Appendix C: Kia / Hyundai API Vulnerabilities (2022-2023)
- Appendix D: Software Supply Chain Security
- Appendix E: Software Defined Vehicle (SDV) Security
- Appendix F: Advanced AI/ML Security
- Appendix G: Cybersecurity Tool Reference Guide
- Appendix H: Maturity Model (Extended)
- Appendix I: Glossary (90+ terms)
- Appendix J: Post-Quantum Cryptography Reference

Appendix A: Jeep Cherokee Remote Hack (2015)

A.1 Overview

Researchers Charlie Miller and Chris Valasek demonstrated the first publicly documented remote compromise of a production vehicle. They exploited the Uconnect infotainment system in a Jeep Cherokee over a Sprint cellular network, achieving full remote control of critical vehicle functions while the vehicle was in motion on a highway — with no physical access and no prior vehicle interaction. The research was disclosed to FCA before public presentation at DEF CON 23 (August 2015).

A.2 Attack Chain

Internet (Remote Attacker) → Sprint 3G/4G Cellular Network
→ Uconnect Infotainment System (WiLink chip, TCP port 6667, no authentication)
→ CAN Bus Gateway ECU (insufficient message origin validation)
→ Safety-Critical ECUs (Steering / Brakes / Transmission)

A.3 Vulnerabilities Exploited

Vulnerability	Technical Detail	Impact
Exposed cellular service	Uconnect chip listened on TCP port 6667 with no authentication; reachable from internet via Sprint network	Remote code execution on head unit
Weak network segregation	Infotainment and chassis CAN buses not properly isolated at gateway	Pivot to safety-critical ECUs
Insufficient gateway filtering	Gateway forwarded arbitrary CAN frames without origin validation or MAC	Unauthorized actuator commands
No message authentication	CAN messages lacked CMAC or freshness counters; replay and injection trivial	Steering, braking, transmission control
Debug interfaces active	JTAG/UART left accessible on production PCB, enabling firmware analysis	Aided exploit development and reverse engineering

A.4 Impact Demonstrated

Function	Effect	Safety Severity
Remote steering	Full left/right steering override at low speed	S3 — Life-threatening
Brake control	Brakes applied or disabled remotely	S3 — Life-threatening
Transmission control	Shift to neutral, kill drive at highway speed	S3 — Life-threatening
Engine shutdown	Vehicle stopped remotely on highway	S2 — Severe injuries
Infotainment/climate	Music, A/C, wipers, display	S0 — Nuisance

Function	Effect	Safety Severity
	manipulated	

A.5 Industry Response

- FCA issued a voluntary recall of 1.4 million vehicles — the first software-only vehicle recall in US history.
- NHTSA opened a formal investigation, accelerating federal engagement with automotive cybersecurity.
- FCA patched the Uconnect firmware via USB and later OTA (no anti-rollback initially — itself a follow-up security concern).
- The incident directly influenced NHTSA's 2016 Cybersecurity Best Practices guidance.
- European regulators cited this case during drafting of UNECE R155 requirements for CSMS.

A.6 Mapping to ISO/SAE 21434

21434 Clause / Area	Gap Exposed	Required Work Product
TARA (Clause 15)	Remote cellular entry path not modelled as high-feasibility attack	Attack path analysis with feasibility scoring
Architecture / Design (Clause 10)	No gateway isolation between infotainment and chassis CAN	Trust boundary map, segmentation requirement
Cryptography (Clause 10)	No CAN message authentication (MAC/CMAC)	SecOC integration requirement, key plan
Verification (Clause 11)	No penetration testing of external interfaces pre-production	Pen test evidence, fuzz test results
Operations / Incident Response (Clause 13)	No OTA patch capability; response took weeks	CSMS incident playbook, OTA pipeline

i KEY PRINCIPLE

Key Lesson: Segmentation is mandatory. An internet-reachable infotainment system must never share a network segment with safety-critical ECUs. Attack chains must be modelled end-to-end, not per-interface in isolation.

Appendix B: Tesla Security Program — Industry Benchmark

Tesla has established one of the automotive industry's most mature and publicly visible cybersecurity programs. While other OEMs implement equivalent controls privately, Tesla's transparency and rapid response cadence make it a practical reference model for the industry.

B.1 Security Architecture Overview

Vehicle Hardware Root of Trust (HSM / Secure Enclave)

- PKI — Per-vehicle certificates for identity and update authorization
- Mutual TLS — Vehicle ↔ Cloud API (all production traffic)
- Signed Firmware — All ECU images signed with OEM private key
- OTA Platform — Staged rollout with A/B partition and rollback
- Cloud SOC — Fleet telemetry, ML anomaly detection, incident response

B.2 Key Security Practices

Practice	Description	Security Value
Bug Bounty Program	Researchers rewarded \$100–\$15,000+ per finding; scope covers all vehicle systems including Autopilot	Expands test coverage beyond internal team; builds researcher goodwill; finds attack chains before attackers do
OTA Vulnerability Remediation	Critical patches deployed to entire fleet within days; no dealer involvement required	Closes attacker window fast; consistent fleet security posture across all deployed VINs
Secure Update Signing	A/B partition; anti-rollback; signed manifest; post-install health check before activation	Prevents malicious update injection and firmware downgrade attacks
Security Research Collaboration	Published technical details of fixes; hosted researchers; co-authored CVE advisories	Normalises transparency; raises industry standards; attracts top security talent
Threat Modeling Integration	Internal red team performs attack path analysis on new features before launch	Architectural flaws caught in design phase, not post-production
Hardware Security Module	Per-ECU HSM for key storage, secure boot measurement, and message authentication	Software compromise cannot extract private keys or forge MAC values
Fleet Monitoring	Continuous telemetry from consenting vehicles; ML-based anomaly detection at fleet scale	Detects novel attack patterns across millions of vehicles simultaneously

B.3 Keen Security Lab Disclosures (2016–2019)

Year	Technique	Entry Point	Impact
2016	Browser exploit chain → CAN pivot	In-vehicle browser (WebKit vulnerability)	Brake, door, dashboard control from 12 miles
2017	Autopilot adversarial input	Lane detection camera spoofing with printed	Involuntary lane change while Autopilot active

Year	Technique	Entry Point	Impact
		patterns	
2019	ECU compromise via OBD-II + CAN	Physical OBD port with custom tooling	Firmware extraction, CAN frame injection

All three disclosures were responsible disclosures before public release, demonstrating the value of Tesla's bug bounty and researcher engagement program. Tesla deployed OTA patches within days of disclosure for the 2016 and 2019 findings.

B.4 Lessons for OEMs and Tier-1 Suppliers

- Continuous vulnerability management requires both a process and a technical pipeline — neither alone is sufficient.
- OTA is a strategic security asset, not just a feature delivery mechanism. Without it, patches require costly dealer campaigns taking weeks or months.
- Security by design means threat modelling before architecture is locked, not a security review after design freeze.
- A bug bounty program with wide scope finds attack chains that internal testing consistently misses.
- Transparency about vulnerabilities and patches builds market trust rather than eroding it — OEMs that hide security issues face worse regulatory and reputational outcomes when issues are later exposed.
- Fleet monitoring at scale enables detection of zero-day exploitation across millions of vehicles simultaneously.

i KEY PRINCIPLE

Benchmark Principle: Security credibility is built through consistent action — fast patching, open disclosure, and architectural rigour — not through certification alone.

Appendix C: Kia / Hyundai Connected Vehicle API Vulnerabilities (2022–2023)

Between 2022 and 2023, security researchers discovered and responsibly disclosed critical API authorization vulnerabilities affecting millions of Hyundai and Kia vehicles with connected services. The vulnerabilities allowed unauthorized third parties to remotely control vehicle functions — including unlocking, starting, and tracking — using only a victim's email address.

C.1 Attack Categories and Technical Detail

Attack Category	Technical Root Cause	Functions Exposed	Scale
API Authorization Flaw	Backend APIs did not verify requesting account was authorized for target VIN; BOLA (Broken Object-Level Authorization)	Remote start, stop, lock, unlock, horn, lights	Millions of vehicles (Hyundai MY2012+, Kia MY2017+)
Session Management Weakness	JWT tokens were long-lived; no re-authentication on sensitive actions; no rotation after privilege-sensitive operations	Persistent vehicle control after one-time token capture	All connected vehicle accounts
Identity Verification Gap	Account registration did not verify email ownership; attacker could pre-register victim email	Account hijack enabling all connected services	New account registrations globally
Telematics API Lateral Movement	Dealer and OEM internal APIs shared same authorization model as consumer APIs; dealer tokens reusable for any VIN	Fleet-wide vehicle access via dealer credential compromise	All dealer-connected vehicles

C.2 Attack Chain

Attacker obtains victim email address (publicly available / social media)

- Attacker calls Hyundai/Kia API with victim email as target identifier
- API returns valid session token without checking account ownership of VIN
- Attacker uses token to call vehicle control APIs (unlock, start, locate)
- Vehicle responds to unauthenticated remote commands

C.3 Engineering Takeaways

Principle	Required Control	Standard / Framework
Strong API Authorization	BOLA prevention: every API call must verify authenticated user owns/is authorized for the target resource (VIN/account)	OWASP API Security Top 10 — API1:2023
OAuth 2.0 / OIDC	Use industry-standard	RFC 6749, RFC 9068

Principle	Required Control	Standard / Framework
	authorization frameworks with short-lived tokens, refresh rotation, and scope restriction	
Zero-Trust Backend Design	Treat every API call as untrusted regardless of origin; enforce authorization at each service boundary	NIST SP 800-207 Zero Trust Architecture
API Rate Limiting	Limit VIN lookup or control requests per account per time window to prevent enumeration and bulk attacks	OWASP API Security — API4:2023
Email / Identity Verification	Verify email ownership at account creation via confirmed OTP or signed link before any vehicle association	NIST SP 800-63B
Security Testing of APIs	Include API authorization testing (BOLA, BFLA, token reuse) in every release pen test and CI/CD pipeline	ISO/SAE 21434 Clause 11, OWASP ASVS

I KEY PRINCIPLE

Key Lesson: Vehicle cloud APIs are part of the vehicle attack surface. A compromise of backend API authorization can give an attacker remote physical control of millions of vehicles simultaneously — with no local network access, no malware, and no hardware. Treat vehicle APIs with the same rigour as vehicle ECU firmware.

Appendix D: Automotive Software Supply Chain Security

Modern automotive software stacks contain millions of lines of code from dozens of suppliers and thousands of open-source packages. Supply chain compromise is attractive to attackers because it bypasses vehicle-level perimeter controls entirely — injecting malicious code into trusted software before it is ever installed on a vehicle.

D.1 Modern Supply Chain Risk Taxonomy

Risk Category	Attack Vector	Automotive Example	Detection Difficulty
Open-source dependency compromise	Malicious package to npm/PyPI/Conan; typosquatting; malicious maintainer takeover	Compromised vehicle diagnostic Python library injected into ECU test toolchain	High — often unsigned packages
CI/CD pipeline compromise	Injection of malicious build steps via stolen CI credentials, compromised runner, or poisoned cache	Malicious binary injected into AUTOSAR BSW build without source change	Very High — no source diff
Signing infrastructure attack	Theft of OEM or Tier-1 code signing private key; compromise of signing server HSM	Malicious firmware signed with legitimate OEM key — accepted by all vehicles	Extreme — trusted artifact
Supplier credential compromise	Stolen developer or CI service account used to inject code or push malicious commits	Backdoor merged via compromised Tier-2 supplier GitHub account	High — normal commit workflow
Third-party SDK/library	Vulnerable or backdoored SDK used by multiple OEMs simultaneously	Shared telematics SDK with hidden data exfiltration channel	Medium — auditable but rarely audited
Hardware component	Counterfeit ECU, modified MCU, or rogue firmware pre-installed in logistics chain	Replacement ECU with modified boot sequence bypassing secure boot	Very High — physical inspection required

D.2 XZ Utils Supply Chain Attack (CVE-2024-3094) — Automotive Relevance

The XZ Utils backdoor (discovered March 2024) is the most sophisticated publicly documented software supply chain attack. A threat actor (identified as "Jia Tan") spent approximately two years contributing to the XZ Utils open-source project, building trust with maintainers, before inserting a backdoor in version 5.6.0 that targeted OpenSSH authentication on systemd-based Linux systems.

- **Attack technique:** Social engineering of open-source maintainers over multi-year period; malicious code hidden in build scripts, not in the C source.
- **Automotive relevance:** Linux-based ECUs (HPC nodes, infotainment, TCU) increasingly use systemd; automotive build systems commonly pull from distribution package managers.

- **Detection:** Discovered by coincidence (Andres Freund noticed performance anomaly in SSH timing) — not by supply chain security tooling, illustrating the limits of current tooling.
- **Mitigation lesson:** SBOM alone is insufficient — build provenance, binary attestation (SLSA), and code signing are needed alongside SBOM for supply chain integrity.

D.3 Secure Supply Chain Framework

Supplier — delivers source, binary, or component with signed SBOM + security attestation

- SBOM Validation — compare declared components vs. actual; check CVEs against SBOM
- Build Integrity — reproducible build; locked dependency versions; isolated build runners
- Artifact Signing — HSM-backed signing ceremony; signature policy enforcement (SLSA L3+)
- Integration Testing — security regression tests; fuzz harness for new parser code
- Release Approval — vulnerability gate; SBOM completeness; provenance record
- Fleet Monitoring — SBOM-driven CVE alerting; impact assessment per VIN population

D.4 SLSA Framework for Automotive

SLSA Level	Requirements	Automotive Application
Level 1	Build process documented; provenance available	Minimum for all ECU firmware builds
Level 2	Build service used; provenance signed by build service	Required for safety-relevant software (ASIL B+)
Level 3	Hardened build platform; ephemeral isolated environments	Required for ASIL D / CAL 4 software; OTA signing pipeline
Level 4 (aspirational)	Two-person review; hermetic builds; parameterized build definitions	Target for signing infrastructure itself

D.5 Required Controls by Development Stage

Control	Purpose	Implementation Guidance
SBOM (CycloneDX / SPDX)	Full inventory of components, versions, licenses, and origins for every release artifact	Generated at build time; supplier-provided for binaries; stored per VIN range; updated per patch
Code Signing	Ensure every binary delivered to a vehicle has been explicitly approved by the OEM signing process	HSM-backed private key; offline root signing key; witness signature for release ceremony; SLSA provenance
Static Analysis (SAST)	Detect CWE vulnerability patterns in source code before build	Coverity / Polyspace for embedded C; SonarQube for toolchain; integrated in CI gate as blocking check
Dependency Vulnerability Scanning	Map SBOM components to CVE database; alert on newly published CVEs post-release	OWASP Dependency-Check; Gype; Trivy; integrated with NVD / OSV / GHSA feeds
Supplier Security Audits	Verify suppliers apply security engineering processes equivalent to OEM expectations	ISO/SAE 21434 Clause 7 audits; supplier TARA review; security interface agreements in contracts
Reproducible Builds	Guarantee given source commit always produces identical binary artifact	Hermetic build environments; locked toolchain; build provenance attestation (SLSA Level 3+)

Control	Purpose	Implementation Guidance
Signed Commits + Branch Protection	Prevent unauthorized code from entering main branch without detection	GPG/SSH commit signing; protected main branch; mandatory peer review gate; CI signature check

i KEY PRINCIPLE

Key Lesson: The supply chain is the new perimeter. An OEM that secures its own vehicle perfectly but accepts unverified binaries from suppliers has an unsecured vehicle. SBOM, code signing, and supplier security requirements must be contractual obligations, not optional add-ons.

Appendix E: Software Defined Vehicle (SDV) Security

The Software Defined Vehicle (SDV) paradigm replaces 100+ isolated ECUs with centralized high-performance compute nodes, zonal controllers, and service-oriented software architectures. This architectural shift dramatically changes the attack surface, threat model, and required security controls.

E.1 Architecture Comparison

Dimension	Traditional Architecture	Software Defined Vehicle
ECU count	100+ independent ECUs	Central Vehicle Computer (HPC) + zonal controllers
Software organization	Domain-based, function-coupled	Service-oriented (SOME/IP, DDS), containerized
Hardware relationship	Dedicated hardware per function	Virtual functions sharing compute resources
Connectivity	Limited external connectivity	Cloud-native services, APIs, always-on connectivity
Feature changes	Hardware swap or reflash required	Feature changes via OTA software update
Blast radius (compromise)	Limited to one function ECU	HPC compromise can affect all functions on that node

E.2 New Attack Surfaces in SDV

New Attack Surface	Threat	Traditional Equivalent
Centralized HPC	Single software compromise affects all functions on HPC; OS-level exploit has vehicle-wide blast radius	Isolated domain ECU — compromise limited to one function
Service discovery (SOME/IP/DDS)	Unauthorized service invocation; service enumeration; rogue service registration masquerading as legitimate	Fixed CAN IDs — no discovery protocol
Container / hypervisor platform	Container escape giving access to host OS; hypervisor vulnerability enabling cross-VM lateral movement	Bare-metal ECU — no abstraction layer
Cloud synchronization	Cloud-to-vehicle command injection; backend compromise with fleet-wide control blast radius	No cloud connectivity in traditional ECU
Android Automotive / third-party apps	Browser exploits, third-party app sandbox escape, app store supply chain attack	Fixed firmware — no third-party apps
Remote diagnostics (SOVD/MQTT)	Unauthorized SOVD resource access; session escalation; diagnostic spoofing over IP	OBD-II physical-only access

E.3 SDV Security Stack

Application Layer – App sandboxing, code signing, least-privilege permissions
↓
Service Layer – Mutual TLS, service authentication, SOME/IP access control
↓
Container Layer – Signed images, runtime security policies (OCI), resource isolation
↓
Operating System – MAC (SELinux/AppArmor), kernel hardening, IDS agents (eBPF)
↓
Hypervisor – VM isolation, inter-VM communication policy, secure interrupt handling
↓
Hardware Root of Trust – HSM / Secure Enclave, measured boot, remote attestation

E.4 Key Security Controls for SDV

Control	SDV Application
Secure containers	Sign all container images (OCI cosign); verify signature at runtime; enforce image policy; scan for CVEs before deployment
Mutual TLS	All service-to-service communication within vehicle software platform must use mTLS with per-service certificates
Service auth + authorization	Every SOME/IP / DDS service invocation must carry authenticated identity; service registry must enforce caller authorization before dispatching
Runtime monitoring	Deploy eBPF-based runtime security agent to detect anomalous process behavior, unexpected network calls, or privilege escalation attempts
Zero Trust architecture	Never assume a service call is safe because it originates from inside the vehicle network; verify identity and authorization at each hop
Hypervisor hardening	Minimize hypervisor attack surface; disable unused para-virtualization features; enforce strict inter-VM memory isolation; no shared memory pages
OTA for all layers	Container images, OS packages, hypervisor, ML models, and firmware must all be updatable via signed OTA pipeline — not just ECU firmware

i KEY PRINCIPLE

SDV Security Principle: Consolidation increases efficiency but concentrates risk. Every architectural consolidation decision must be accompanied by equivalent consolidation of security controls — segmentation, monitoring, and blast-radius planning.

Appendix F: Advanced AI / ML Security for Automotive Systems

AI and machine learning are deeply integrated into modern vehicles: ADAS perception, sensor fusion, predictive diagnostics, voice assistants, and anomaly detection all rely on ML models. Each creates a distinct attack surface that does not exist in traditional rule-based systems.

F.1 AI Threat Categories

Threat Category	Description	Automotive Example	TARA Impact
Adversarial ML	Crafted input perturbations — imperceptible to humans — that cause deployed model to produce wrong outputs	Stop sign with adversarial patches causes ADAS to classify it as speed-limit sign; vehicle fails to stop	S3, CAL 4
Data Poisoning	Corrupting training data so model learns incorrect behavior that persists silently in production	Fleet data collection pipeline poisoned to bias lane-keeping model toward unsafe positions	S2–S3, CAL 3–4
Model Extraction / Theft	Black-box query attacks that reconstruct proprietary model from outputs	Competitor extracts OEM's proprietary sensor fusion model	Financial, CAL 2
Model Inversion	Recovering training data from deployed model outputs, exposing private driver data	Driving style model reveals individual route history or biometric patterns	Privacy, CAL 2
Prompt Injection (GenAI)	Injecting adversarial instructions into GenAI assistant inputs via third-party content	Navigation POI data contains embedded prompt causing AI assistant to execute unauthorized commands	S1–S2, CAL 2–3
Model Update Tampering	Replacing legitimate ML model update with poisoned or backdoored version	OTA model update for ADAS perception swapped with adversarially trained variant	S3, CAL 4

F.2 Adversarial ML — Technical Detail

- Physical-world adversarial attacks use printed patterns, stickers, or lighting changes visible to cameras but classified incorrectly — Eykholt et al. (2018) demonstrated stop sign misclassification with printed patches.
- White-box attacks (FGSM, PGD, C&W) require full model access; black-box attacks require only query access to model outputs — the latter is more relevant to deployed vehicle systems.
- Ensemble defenses, adversarial training, and certified robustness (randomized smoothing) increase resistance but typically at a cost to clean-input accuracy.
- Detection of adversarial inputs using statistical tests (feature squeezing, input randomization) can be layered in front of the primary model.

- **Formal certification of ML robustness remains an open research problem as of June 2026.** Current SOTIF (ISO 21448:2022) guidance requires monitoring and conservative design margins for safety-relevant ML functions.

F.3 AI Security Lifecycle

Phase	Security Activity	Key Artifact
Data Collection	Audit data sources for poisoning risk; enforce data provenance tracking; apply differential privacy where appropriate	Data provenance log, privacy impact assessment, data quality certification
Training	Monitor training for unexpected accuracy changes; use holdout adversarial test sets; validate on OOD samples	Adversarial evaluation report, training audit log, OOD validation results
Validation	Run robustness benchmarks; test against known attack families (FGSM, PGD, C&W); validate on real-world edge cases	Robustness report, edge-case test results, SOTIF validation evidence
Deployment	Sign model weights and configuration; verify signature at load time; deploy through SUMS-compliant OTA pipeline	Signed model artifact, OTA deployment record, version control evidence
Monitoring	Monitor model performance in production; detect distribution shift and inference anomalies; alert on unexpected patterns	Model health dashboard, anomaly alert log, drift detection report
Retraining	Treat retraining as a new development cycle; apply full validation pipeline before OTA deployment of updated model	Retraining change record, updated validation report, SUMS campaign evidence

F.4 AI-Based IDS Architecture

Vehicle Signals (CAN, Ethernet, GPS, Sensor Streams)

- ↓ Feature Extraction (message rate, timing, payload statistics, inter-ECU patterns)
- ↓ ML Model (anomaly detection: Autoencoder / Isolation Forest / LSTM / Transformer)
- ↓ Risk Scoring (anomaly score weighted by asset ASIL / CAL level)
- ↓ Alert Generation (threshold-based trigger → IDS event with context + confidence)
- ↓ Backend Correlation (fleet-level aggregation → novel attack detection across VINs)

1 KEY PRINCIPLE

AI Security Principle: ML models deployed in safety-relevant functions must be treated as safety-critical software under ISO 26262/SOTIF AND cybersecurity-critical under ISO/SAE 21434. Model updates must follow SUMS. Model integrity must be verified at runtime load time.

Appendix G: Cybersecurity Tool Reference Guide

This appendix catalogues the principal tools used across the automotive cybersecurity engineering lifecycle — from threat modelling and secure development through vehicle network analysis, penetration testing, fuzzing, and security operations.

G.1 Threat Modelling Tools

Tool	Vendor Origin	Primary Usage	Notes
Microsoft Threat Modeling Tool	Microsoft	DFD-based STRIDE threat elicitation	Free; good for cloud and IT; less automotive-specific
OWASP Threat Dragon	OWASP	Open-source DFD and STRIDE modelling; CI/CD integration	JSON-based models; GitHub integration; good for teams
IriusRisk	Iriusrisk Ltd	Automated threat and countermeasure generation from templates	Commercial; strong automotive / embedded templates
Yakindu Security Analyst	Itemis AG	Eclipse-based automotive security analysis; TARA automation	Strong ISO 21434 alignment; TARA workflow integration
ThreatModeler	ThreatModeler Inc.	Cloud and embedded threat modelling with STRIDE/LINDDUN	Commercial; API-driven for DevSecOps pipelines

G.2 Vehicle Network Analysis Tools

Tool	Vendor	Protocol Support	Typical Use Case
CANoe	Vector Informatik	CAN, CAN-FD, LIN, Ethernet, SOME/IP, DoIP	ECU simulation, network test, gateway validation, AUTOSAR testing
CANalyzer	Vector Informatik	CAN, CAN-FD, LIN, Ethernet	Bus monitoring, message logging, filter analysis, injection testing
Vehicle Spy 3	Intrepid Control Systems	CAN, CAN-FD, LIN, Ethernet, FlexRay	Vehicle diagnostics, data logging, network node simulation
Wireshark + Automotive Dissectors	Wireshark Foundation	DoIP, SOME/IP, Ethernet/IP	Automotive Ethernet protocol analysis; passive capture and replay
can-utils (socketcan)	Linux community	CAN (Linux SocketCAN)	Low-level CAN frame capture, injection, and replay on Linux
OpenXC Platform	Ford / Open Source	OBD-II / CAN	Vehicle data acquisition and research; JSON-based vehicle data API

G.3 Penetration Testing Tools

Tool	Category	Automotive Application
------	----------	------------------------

Tool	Category	Automotive Application
Metasploit Framework	Exploitation framework	Exploiting known CVEs in telematics unit OS, ECU web services, V2X infrastructure
Burp Suite Pro	Web / API testing	Vehicle mobile apps, cloud backend APIs, OTA distribution endpoints, BOLA/BFLA testing
Kali Linux	Penetration testing OS	Full toolset environment for automotive pen testing; includes 600+ security tools
Scapy (Python)	Packet crafting	Custom automotive protocol packet crafting for CAN, DoIP, SOME/IP, V2X testing
GDB + PEDA	Debugger / exploit dev	Dynamic analysis of ECU firmware; finding memory corruption vulnerabilities
Frida	Dynamic instrumentation	Runtime hooking of Android Automotive / QNX / Linux processes for ECU and IVI analysis
Nmap + NSE Scripts	Network scanning	Discovering open services on vehicle Ethernet interfaces and telematics units
OWASP ZAP	Web/API scanner	Automated scanning of vehicle cloud APIs; DAST for connected service portals

G.4 Fuzz Testing Tools

Tool	Technique	Automotive Use Case
AFL++ (American Fuzzy Lop++)	Coverage-guided mutation fuzzing	ECU firmware parsers, OTA package decoders, diagnostic message handlers; C/C++ targets
libFuzzer	In-process coverage-guided fuzzing	Parser libraries integrated directly in build system; fast, low overhead; LLVM-integrated
Boofuzz	Network protocol fuzzing	DoIP, SOME/IP, UDS over IP service fuzzing; Python-based; session-aware state machine support
Peach Fuzzer	Model-based protocol fuzzing	Structured protocol fuzzing for CAN, UDS, AUTOSAR service interfaces; PIT language
Jazzer (JVM)	Coverage-guided Java fuzzing	Android Automotive app fuzzing; Java-based telematics middleware testing

G.5 Static Analysis and Secure Coding Tools

Tool	Language	Strength	Automotive Use
Coverity SAST	C/C++, Java, Python	Deep inter-procedural analysis; CWE mapping; low false positive rate	ECU firmware, gateway software, AUTOSAR stack analysis
Polyspace Bug Finder / Code Prover	C/C++, Ada	Formal methods-based; MISRA/AUTOSAR coding rule enforcement; ASIL support	Safety-critical and security-critical embedded C per ISO 26262
SonarQube	25+ languages	Security hotspot detection; code quality; CI/CD integration	Toolchain code, Python/Java backend services, CI pipeline enforcement
CodeChecker (LLVM)	C/C++	Open-source; Clang-Tidy +	Open-source alternative for

Tool	Language	Strength	Automotive Use
		Clang Static Analyzer; fast; free	ECU C/C++ analysis; CI gate
Semgrep	Multi-language	Pattern-based SAST; custom rule authoring; fast scanning	Custom rules for automotive-specific security patterns; Python tooling

G.6 Security Monitoring and SOC Tools

Tool	Category	Automotive Application
Splunk Enterprise Security	SIEM / SOC platform	Fleet-level vehicle security event aggregation, correlation, and threat hunting
ELK Stack (Elasticsearch + Logstash + Kibana)	Log analytics platform	Open-source fleet log aggregation; custom automotive dashboards; ML anomaly detection
Microsoft Sentinel	Cloud-native SIEM	Cloud-connected vehicle telemetry analysis; integration with Azure IoT Hub
Argus Cyber Security Platform	Automotive IDS/SOC	Purpose-built for automotive; in-vehicle IDS + backend SOC; AUTOSAR and Linux support
Karamba Security XGuard	Automotive ECU runtime protection	ECU-level runtime integrity enforcement; process whitelisting; anomaly detection at ECU
UpGuard / BitSight	Third-party risk	Supplier cybersecurity posture monitoring; supply chain risk scoring

i KEY PRINCIPLE

Tool Selection Principle: Choose tools that integrate into the development pipeline, not just run at release. SAST in CI, fuzzing in nightly builds, and fleet monitoring at SOP are more effective than point-in-time audits before launch.

Appendix H: Automotive Cybersecurity Maturity Model

Building automotive cybersecurity capability is a multi-year organisational journey. This appendix provides a five-level maturity model with detailed capability descriptions, observable indicators, and a practical advancement roadmap aligned with ISO/SAE 21434 CSMS requirements and UNECE R155 audit expectations.

H.1 Maturity Level Definitions

Level	Name	Characteristics	Observable Indicators	Typical Gap
1	Ad Hoc	Security via individual heroics; no repeatable process; reactive patching; limited threat awareness	No TARA; security found in pen test at release; no SBOM; no incident playbook; no security training	No process; outcomes depend entirely on individuals
2	Repeatable	Basic process documented; some ISO 21434 alignment; security requirements written; supplier questionnaires	TARA exists but incomplete; basic pen test; some secure coding guidelines; security in design reviews	Inconsistent execution; traceability gaps; supplier compliance assumed not verified
3	Managed	Metrics-driven; formal CSMS; cross-functional governance; evidence produced and audited	Complete TARA with risk decisions; CAL requirements with test evidence; IDS deployed; patch process	Limited automation; fleet monitoring immature; supplier audit coverage partial
4	Optimized	Continuous improvement; automation in testing and monitoring; proactive threat intelligence; fleet learning	CI/CD security gates; automated SBOM and CVE scanning; fleet IDS with ML; MTTD/MTTR tracked	Scaling to growing fleet and SDV architectural complexity
5	Resilient	Fast incident response; strong supply chain trust; proactive security design; industry information sharing	Sub-24h critical patch OTA; Auto-ISAC participation; AI-driven threat detection; regulatory engagement	Maintaining security edge as threats and vehicle technology evolve simultaneously

H.2 Capability Assessment by Domain

Domain	Level 1	Level 2	Level 3	Level 4 / 5
Governance	No security roles or policies	Security policy documented; roles assigned	CSMS certified; security steering; metrics	Integrated with risk management; continuous improvement

Domain	Level 1	Level 2	Level 3	Level 4 / 5
TARA / Risk Engineering	No TARA; ad hoc risk discussions	TARA done for some items; no formal method	Full TARA per 21434; CAL assigned; linked to design	TARA automated; updated continuously; fleet-data driven
Architecture	No security architecture; trust boundaries undocumented	Basic segmentation; some secure design patterns	Formal trust model; gateway controls; HSM integration	Zero Trust; runtime verification; SDV security stack
Verification	Release-only pen test; no regression	SAST in CI; basic fuzz; pen test at SOP	Full pyramid: SAST + unit + fuzz + HIL + pen test	Automated regression; adversarial ML testing; red team
Supply Chain	Supplier security assumed; no SBOM	Supplier questionnaires; basic security clauses	SBOM required; 21434 Clause 7 audits; TARA review	Real-time CVE monitoring; supplier scoring; SLSA L3+
Operations	No monitoring; manual patching	IDS deployed; basic alerting; patch process exists	Fleet SOC; MTTD/MTTR tracked; incident playbooks	AI-driven anomaly; auto patch scoring; predictive analytics

H.3 Key Performance Indicators by Level

KPI	L2 Target	L3 Target	L4 Target	L5 Target
TARA coverage (% assets with completed TARA)	50%	100%	100% + continuous update	100% + fleet-data driven
CAL requirements with test evidence	<50%	>90%	100% with CI traceability	100% automated
Supplier SBOM delivery rate	<25%	>75%	>95%	100% with CVE alerting
Mean Time to Detect (MTTD) — critical threat	Weeks	Days	Hours	Minutes (automated)
Mean Time to Recover (MTTR) — critical patch	Months (dealer)	Weeks (OTA)	Days (OTA)	<24 hours (OTA)
Security training completion (engineering)	<50%	>80%	>95%	100% + advanced track
Pen test critical findings closed before SOP	<50%	>90%	100%	100% + pre-SOP red team

i KEY PRINCIPLE

Maturity Principle: Every organisation is at Level 1 at some point. The goal is not immediate perfection but continuous, measurable progress. Honest self-assessment is the prerequisite for advancement. Declare the current level accurately, fund the gap, and own the timeline.

Appendix I: Automotive Cybersecurity Glossary

This glossary defines key terms used in automotive cybersecurity engineering, standards, and operations. Terms are aligned with ISO/SAE 21434:2021, UNECE R155/R156, and related standards.

Term	Definition
A/B Partitioning	Update strategy where two independent software slots (active and inactive) allow safe updates with automatic rollback on failure. Only the inactive slot is written during an update.
ASLR (Address Space Layout Randomization)	OS security feature that randomizes memory addresses of key program data areas, making it harder for attackers to predict target addresses for exploitation.
ASIL (Automotive Safety Integrity Level)	Risk classification in ISO 26262 from QM (no specific safety requirement) through ASIL A, B, C, to ASIL D (highest integrity). Determines development rigor required.
Attack Feasibility	TARA parameter assessing how difficult an attack is, based on required access, knowledge, tools, time, and window of opportunity. Scored 1 (highest) to 4 (lowest) in ISO/SAE 21434.
Attack Path Analysis	Systematic mapping of the steps an attacker must complete to reach and exploit an asset, used in TARA to assess feasibility and identify required controls.
Auto-ISAC	Automotive Information Sharing and Analysis Center. Industry body for sharing automotive cybersecurity threat intelligence between OEMs, suppliers, and technology providers.
AUTOSAR (AUTomotive Open System ARchitecture)	Standardized automotive software architecture. Classic AUTOSAR for traditional ECUs; Adaptive AUTOSAR for high-performance compute platforms.
BOLA (Broken Object-Level Authorization)	API vulnerability where a caller can access or modify objects belonging to another user by manipulating resource identifiers. Top automotive cloud API vulnerability category.
CAL (Cybersecurity Assurance Level)	Risk-based rigor classification in ISO/SAE 21434 from CAL 1 (lowest) to CAL 4 (highest), analogous to ASIL in functional safety. Determines required testing and development depth.
CAN (Controller Area Network)	Robust automotive bus standard for inter-ECU communication. No native authentication or encryption; security relies on gateway filtering, MAC, and IDS.
CMAC (Cipher-based Message Authentication Code)	Cryptographic MAC using AES in CBC mode. Used in AUTOSAR SecOC for CAN message authentication. Provides both integrity and authenticity verification.
CSIRT (Computer Security Incident Response Team)	Dedicated team responsible for coordinating detection, response, containment, and recovery from automotive cybersecurity incidents.
CSMS (Cyber Security Management System)	Organizational management system required by UNECE R155, covering processes, governance, and evidence for managing cybersecurity risk across the vehicle lifecycle.
CVE (Common Vulnerabilities and Exposures)	Public database of known cybersecurity vulnerabilities. Used with SBOM for impact assessment: which ECUs and VINs are affected by a given CVE.
CVSS (Common	Industry standard for rating the severity of vulnerabilities (0–10). Context

Term	Definition
Vulnerability Scoring System)	scoring (exploitability in-vehicle, ASIL relevance) is required alongside raw CVSS for automotive triage.
Data Poisoning	Supply chain or adversarial attack that corrupts training data to cause an ML model to learn incorrect behavior that persists silently in production deployment.
Defense in Depth	Security principle using multiple independent layers of controls so that compromise of any single control does not result in full system compromise.
DPIA (Data Protection Impact Assessment)	Privacy engineering artifact required by GDPR Article 35 for high-risk processing. Documents data flows, purposes, risks, and mitigations for vehicle telemetry and user data.
DoIP (Diagnostics over Internet Protocol)	Automotive diagnostic protocol (ISO 13400) enabling UDS diagnostics over Ethernet. Requires authentication and access control to prevent unauthorized diagnostic sessions.
ECU (Electronic Control Unit)	Specialized embedded controller managing one or more vehicle functions. Modern vehicles contain dozens to hundreds of ECUs organized in domains or zones.
FGSM (Fast Gradient Sign Method)	Adversarial ML attack technique that computes gradient of model loss with respect to input and perturbs the input in the gradient direction to fool the model.
FlexRay	High-reliability automotive bus standard providing deterministic timing and fault tolerance, used in X-by-wire applications (steer-by-wire, brake-by-wire).
GDPR (General Data Protection Regulation)	EU regulation governing personal data processing. Vehicles collecting location, driving behavior, and in-cabin data are subject to GDPR; Article 25 requires Privacy by Design.
HARA (Hazard Analysis and Risk Assessment)	ISO 26262 method for identifying hazardous events and assigning ASIL levels. Inputs safety-critical function list that feeds TARA asset identification.
HPC (High-Performance Compute Node)	Centralized compute platform in SDV architectures running multiple VMs or containers. Higher blast radius than single-function ECU if compromised.
HSM (Hardware Security Module)	Dedicated hardware for secure cryptographic operations and key storage. Provides protection against software extraction of private keys even under OS compromise.
IDS (Intrusion Detection System)	System monitoring vehicle network traffic and events to detect and alert on anomalous or malicious activity. May be rule-based, anomaly-based, or hybrid.
ISO 15118	Standard for vehicle-to-grid (V2G) communication. ISO 15118-2 covers AC/DC charging with Plug&Charge; ISO 15118-20 (2022) adds WPT and bidirectional power flow.
JTAG (Joint Test Action Group)	Industry standard for debugging embedded systems, providing full control over processor and memory. Must be disabled or secured in production vehicle ECUs.
MAC (Message Authentication Code)	Cryptographic tag appended to a message to verify its integrity and authenticity. Used in AUTOSAR SecOC (AES-CMAC) and TLS (HMAC).
ML-DSA (Module-Lattice Digital Signature Algorithm)	NIST FIPS 204 post-quantum digital signature standard, finalized August 2024. ReplacesCRYSTALS-Dilithium. Recommended for firmware signing in systems with>10-year service life.
ML-KEM (Module-Lattice Key Encapsulation)	NIST FIPS 203 post-quantum key encapsulation standard, finalized August 2024. Replaces CRYSTALS-Kyber. Recommended for quantum-resistant key exchange in new automotive platforms.

Term	Definition
Mechanism)	
MPU (Memory Protection Unit)	Hardware mechanism partitioning memory into protected regions, preventing unauthorized access between software components running on the same processor.
MTTD (Mean Time to Detect)	Average time between a security event occurring and the team detecting it. Key operational metric for IDS effectiveness; target: minutes for critical threats at L4/L5 maturity.
MTTR (Mean Time to Recover)	Average time between detecting a security incident and fully restoring service. Key operational metric; target: <24 hours for critical patches at L4/L5 maturity via OTA.
OBD-II (On-Board Diagnostics II)	Standardized vehicle diagnostic port (SAE J1979). Physical access point for diagnostic tools; requires strong session authentication to prevent unauthorized ECU access.
OTA (Over-The-Air update)	Wireless software update mechanism for vehicle ECUs. Security requires signed packages, anti-rollback, A/B partitioning, and integrity verification. Mandated governance by UNECE R156.
PGP (Pretty Good Privacy)	Asymmetric cryptographic standard used for commit signing, package signing, and developer identity verification in secure software development workflows.
PKI (Public Key Infrastructure)	System for issuing, distributing, and managing digital certificates. Provides cryptographic identity for vehicles, ECUs, cloud services, and V2X pseudonyms.
PnC (Plug&Charge)	ISO 15118 feature enabling automatic authentication and billing authorization between EV and charging station without user action, using vehicle's contract certificate.
Post-Quantum Cryptography (PQC)	Cryptographic algorithms resistant to attacks by quantum computers. NIST finalized FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) in August 2024.
Privacy by Design	GDPR Article 25 principle requiring privacy to be incorporated into system design from the start, not added as an afterthought. Mandatory for vehicle data collection systems.
R155	UNECE UN Regulation No. 155. Mandates Cyber Security Management System (CSMS) for vehicle type approval in signatory markets including EU, Japan, South Korea, India.
R156	UNECE UN Regulation No. 156. Mandates Software Update Management System (SUMS) for controlled software update delivery including OTA and workshop updates.
SBOM (Software Bill of Materials)	Machine-readable inventory of all software components, versions, licenses, and origins in a software artifact. CycloneDX and SPDX are the primary automotive-applicable formats.
SecOC (Secure Onboard Communication)	AUTOSAR standard module providing message authentication (MAC + freshness counter) for CAN and Ethernet PDUs. Updated in R24-11 to support AES-256-GCM.
SLSA (Supply-chain Levels for Software Artifacts)	Open framework for software supply chain integrity. Level 3 requires hermetic builds, signed provenance, and isolated build environments. Recommended for CAL 4 software components.
SLH-DSA (Stateless Hash-based Digital Signature Algorithm)	NIST FIPS 205 post-quantum signature standard, finalized August 2024. Replaces SPHINCS+. Hash-based; slower than ML-DSA but based on minimal security assumptions.

Term	Definition
SOME/IP (Scalable service-Oriented MiddlewarE over IP)	AUTOSAR service-oriented communication protocol over Automotive Ethernet. Requires authentication and access control to prevent unauthorized service invocation.
SOTIF (Safety of the Intended Functionality)	ISO 21448:2022 standard addressing safety risks from performance limitations or foreseeable misuse without malfunction. Critical for ADAS and autonomous driving; intersects with cybersecurity for adversarial ML.
SOVD (Service-Oriented Vehicle Diagnostics)	Next-generation automotive diagnostic standard (ASAM SOVD) enabling HTTP/REST-based diagnostics over Ethernet. Requires OAuth 2.0 authorization and TLS 1.3 encryption.
SUMS (Software Update Management System)	UNECE R156-required organizational process and toolchain controlling safe software update delivery, including OTA campaigns, compatibility checks, and anti-rollback enforcement.
TARA (Threat Analysis and Risk Assessment)	ISO/SAE 21434 structured method for identifying cybersecurity assets, threat scenarios, attack paths, risk values, and treatment decisions. Core engineering tool for automotive cybersecurity.
TCU (Telematics Control Unit)	ECU providing vehicle connectivity to cellular networks, cloud services, and mobile apps. Primary external attack surface due to persistent internet-reachable interface.
TLS 1.3	Transport Layer Security version 1.3 (RFC 8446). Required for all vehicle-cloud communications; provides forward secrecy, AEAD encryption, and reduced handshake latency.
UART (Universal Asynchronous Receiver-Transmitter)	Serial communication interface used for debugging embedded systems. Must be disabled or physically secured in production ECUs to prevent firmware extraction.
UDS (Unified Diagnostic Services)	ISO 14229 standard for automotive diagnostic communication. Defines diagnostic sessions, memory read/write, and ECU programming services requiring strong access control.
V2G (Vehicle-to-Grid)	Bidirectional energy flow between EV and power grid enabled by ISO 15118-20. Creates billing, authentication, and network entry-point security requirements.
V2X (Vehicle-to-Everything)	Umbrella term for vehicle communication with infrastructure, other vehicles, pedestrians, and networks. Secured by PKI pseudonym certificates (IEEE 1609.2 / ETSI TS 103 097).
VSOC (Vehicle Security Operations Center)	Backend facility monitoring vehicle fleet security events, correlating alerts, managing incident response, and tracking patch campaign deployment.
VIN (Vehicle Identification Number)	Unique 17-character identifier for each vehicle. Used for targeting vulnerability patches, rollout cohort assignment, and incident impact scope assessment.
Zero Trust	Security model where no entity — inside or outside the network — is implicitly trusted. Every request is authenticated, authorized, and verified. NIST SP 800-207 defines the architecture.

Appendix J: Post-Quantum Cryptography Standards Reference (FIPS 203/204/205)

NIST finalized three post-quantum cryptography (PQC) standards in August 2024. This appendix provides a reference guide for automotive security engineers planning PQC migration for long-lived vehicle systems.

J.1 Why PQC Matters for Automotive

- Vehicles deployed today may remain in service for 10–20 years, potentially overlapping with the practical availability of cryptographically relevant quantum computers.
- Current best estimates (NIST, NSA) suggest that "harvest now, decrypt later" attacks — capturing today's encrypted traffic for future quantum decryption — are a real threat for long-lived secrets.
- Code signing keys, PKI private keys, and long-term vehicle identity credentials are particularly at risk if protected only by RSA or classical ECC.
- The automotive industry's 5–7 year vehicle development cycles mean that PQC migration planning must begin now for platforms that will launch in 2028–2032.

J.2 NIST PQC Final Standards (August 2024)

Standard	Algorithm	Replaces	Primitive	Automotive Use Cases
FIPS 203	ML-KEM (Module-Lattice Key Encapsulation Mechanism)	CRYSTALS-Kyber	Lattice-based (Module LWE)	Key encapsulation for TLS sessions, OTA package key wrapping, ECU-to-cloud key establishment
FIPS 204	ML-DSA (Module-Lattice Digital Signature Algorithm)	CRYSTALS-Dilithium	Lattice-based (Module LWE/SIS)	Firmware signing, OTA manifest signing, PKI certificate signing, V2X certificate signing
FIPS 205	SLH-DSA (Stateless Hash-based Digital Signature Algorithm)	SPHINCS+	Hash-based (stateless)	Code signing root keys, long-lived trust anchors, highest security applications

J.3 Algorithm Comparison for Automotive Contexts

Property	Classical P-256	ECC	ML-KEM-768 (FIPS 203)	ML-DSA-65 (FIPS 204)	SLH-DSA-SHA2-128f (FIPS 205)
Public key size	64 bytes		1,184 bytes	1,952 bytes	32 bytes
Private key size	32 bytes		2,400 bytes	4,032 bytes	64 bytes
Signature size	64 bytes		N/A (KEM)	3,293 bytes	17,088 bytes
Security level	Classical 128-bit		NIST Level 3 (~AES-192)	NIST Level 3 (~AES-192)	NIST Level 1 (~AES-128)
Sign/verify speed	Fast		Fast (encaps/decaps)	Fast	Sign: slow; Verify: moderate

Property	Classical P-256	ECC	ML-KEM-768 (FIPS 203)	ML-DSA-65 (FIPS 204)	SLH-DSA- SHA2-128f (FIPS 205)
CAN-FD feasibility	Feasible (64B key)		Not feasible (1.2KB key)	Not feasible (3.3KB sig)	Not feasible (17KB sig)
Automotive Ethernet	Yes		Yes (TLS 1.3)	Yes (OTA manifest)	Yes (offline root signing)
Quantum security	No (Shor's algorithm)		Yes (FIPS 203)	Yes (FIPS 204)	Yes (FIPS 205)

J.4 PQC Migration Strategy for Automotive

Phase 1: Inventory and Assessment (2024–2026)

- Inventory all cryptographic assets: keys, algorithms, protocols, and their locations in vehicle, backend, and supplier systems.
- Classify by urgency: long-lived signing keys>session encryption>ephemeral keys.
- Identify systems with>10-year service life that are candidates for PQC migration in current development cycles.

Phase 2: Hybrid Deployment (2026–2028)

- Deploy hybrid classical + PQC for key exchange in new platform TLS stacks (ECDH + ML-KEM in parallel).
- Hybrid mode provides protection if PQC algorithms have unforeseen weaknesses — both must be broken for the session to be compromised.
- Begin testing ML-DSA for OTA manifest signing on new platforms; maintain ECDSA as fallback.

Phase 3: PQC Primary (2028–2030)

- ML-KEM as primary key encapsulation for all new vehicle-cloud TLS connections.
- ML-DSA as primary algorithm for firmware signing and OTA manifests on new vehicle programmes.
- SLH-DSA for root signing keys (offline use; performance not critical).

Phase 4: Legacy Migration (2030+)

- Classical-only systems require risk acceptance if they cannot receive PQC updates via OTA.
- V2X certificate infrastructure will require industry-wide coordinated migration (ETSI, IEEE standards bodies).

J.5 Key Deployment Guidance

Context	Recommended Approach
TLS 1.3 (vehicle ↔ cloud)	Hybrid: X25519 + ML-KEM-768 key exchange (draft-ietf-tls-hybrid-design); ML-DSA-65 for server certificates in new deployments
OTA firmware signing	ML-DSA-65 or ML-DSA-87 for new platforms; SLH-DSA-SHA2-256s for offline root signing key (rarely used; performance acceptable)

Context	Recommended Approach
CAN SecOC (AUTOSAR)	AES-256-CMAC remains appropriate; PQC not yet practical for CAN bandwidth constraints; classical symmetric crypto is quantum-resistant at 256-bit
V2X pseudonym certificates (ETSI/IEEE)	Industry coordination required; ETSI/IEEE working groups developing PQC profiles for 1609.2 and TS 103 097; monitor and plan migration
ECU secure boot	SLH-DSA for immutable root of trust (offline key; performance acceptable for boot-time use); ML-DSA for application signing keys
PKI certificates (OEM CA)	Begin issuing hybrid certificates for new programmes; plan root CA migration to ML-DSA before classical CAs expire

Uses Open-source data only, Any OEM details or closed IPs not used.